

Taller de Lenguajes de Programación

Sesión 04:

Componentes en Django

Introducción

En esta sesión construiremos las páginas web del Sistema de Gestión de Encomiendas utilizando los componentes frontales de Django: URLs, vistas, templates, formularios, el Admin site y autenticación de usuarios. Al terminar tendremos un sistema web funcional que corre completamente dentro de Docker.

- 4.1. URLs, Views y Templates
- 4.2. Formularios
- 4.3. Django Admin Site
- 4.4. Django Authentication
- 4.5. Sesiones en Django

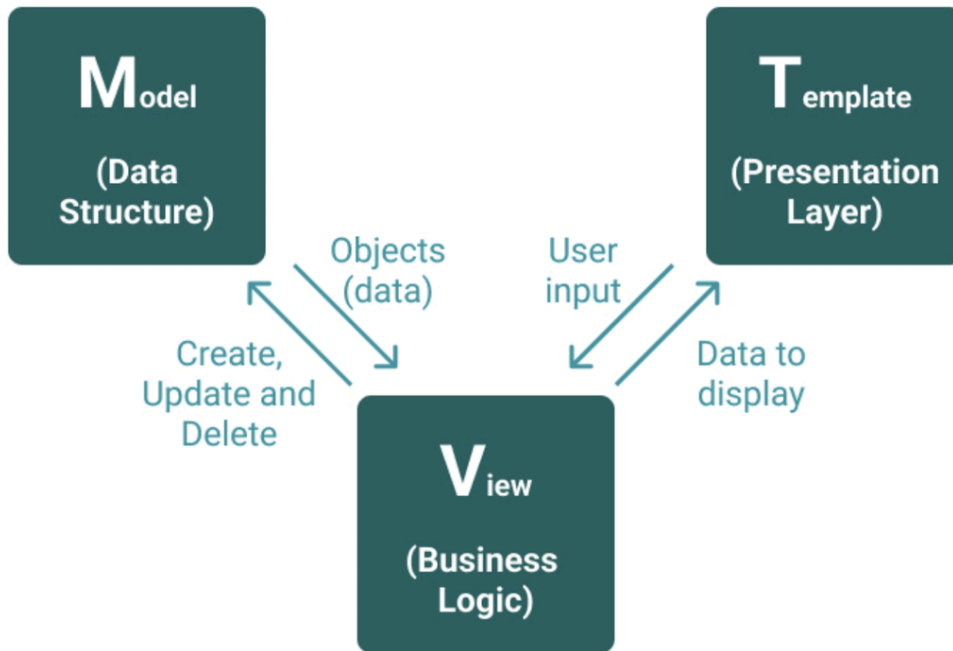
CONTEXTO DEL PROYECTO

Construiremos las siguientes pantallas del sistema de encomiendas:

- Dashboard principal con estadísticas (encomiendas activas, en tránsito, con retraso).
- Listado y detalle de encomiendas con paginación y búsqueda.
- Formulario de registro de nueva encomienda.
- Páginas de login y perfil del empleado.
- Todo protegido con autenticación: solo usuarios con sesión pueden acceder.

URLs, Views y Templates

El modelo MVT (Model-View-Template) en Django



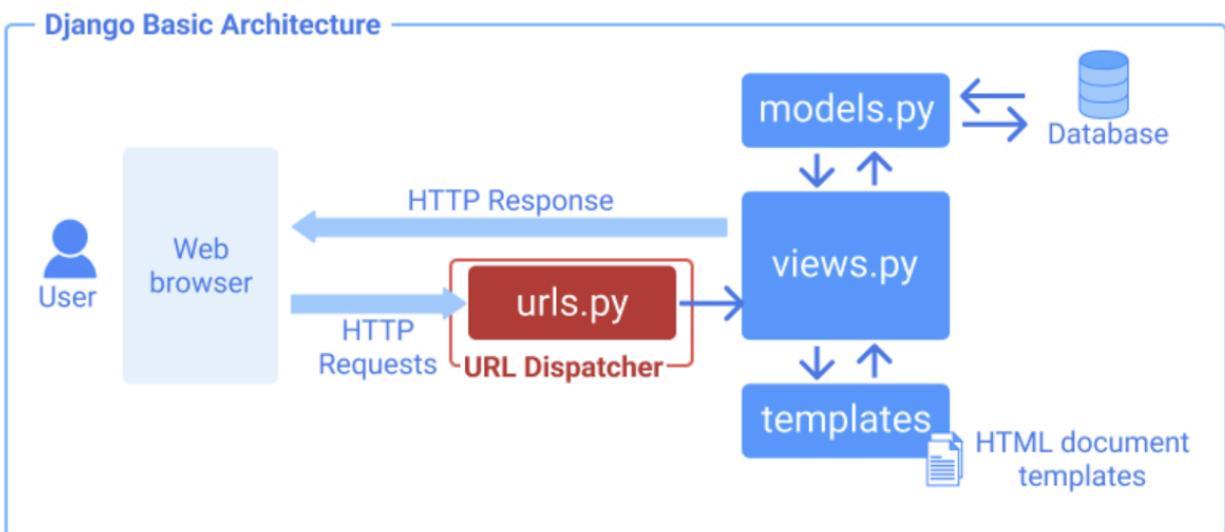
Django se basa en la arquitectura MVT (Model-View-Template). MVT es un patrón de diseño de software para desarrollar una aplicación web.

La estructura MVT tiene las siguientes tres partes -

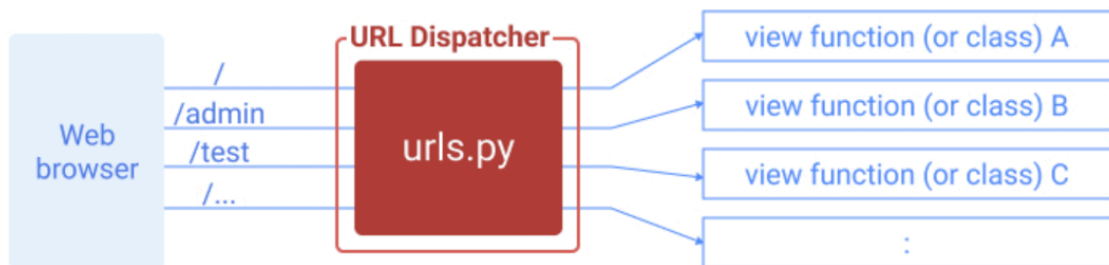
- **Modelo:** El modelo va a actuar como la interfaz de tus datos. Es responsable de mantener los datos. Es la estructura lógica de datos detrás de toda la aplicación y está representada por una base de datos (generalmente bases de datos relacionales como MySQL, Postgres).
- **View:** La Vista es la interfaz de usuario - lo que ves en tu navegador cuando renderizas un sitio web. Está representada por archivos HTML/CSS/Javascript y Jinja. Para saber más
- **Template:** Una plantilla consiste en partes estáticas de la salida HTML deseada, así como alguna sintaxis especial que describe cómo se insertará el contenido dinámico.

URLs de la app Envios

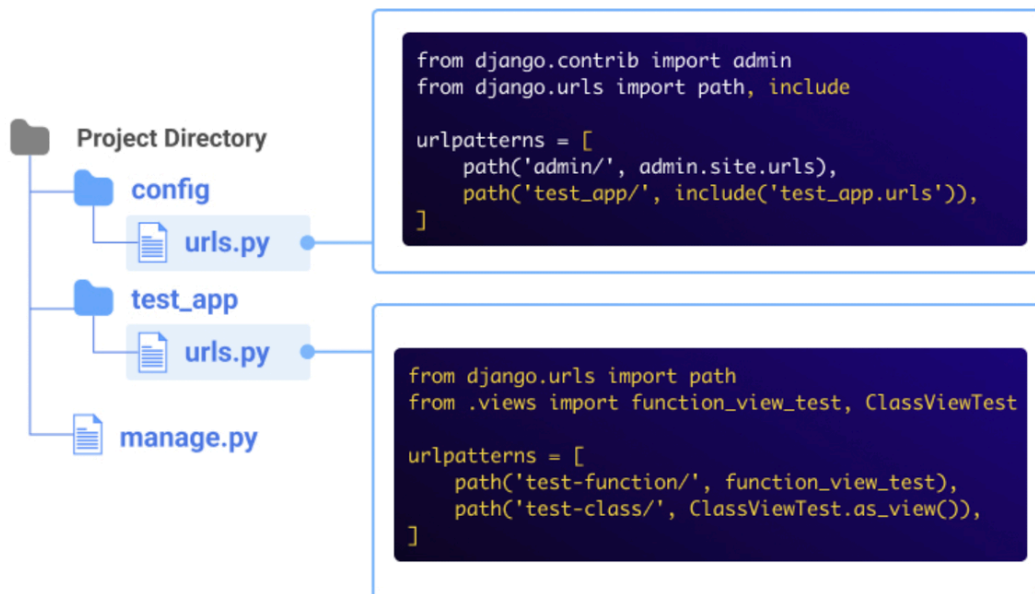
Django utiliza un sistema basado en patrones para definir las rutas de las aplicaciones web. Estas rutas se configuran en archivos llamados `urls.py` y permiten enlazar las URLs con las vistas.



Role of URL Dispatcher



En Django, las rutas se definen en el archivo `urls.py`. La función principal utilizada es `path`, y opcionalmente, `re_path` para expresiones regulares. Cada aplicación debe definir sus rutas,



Para nuestro caso vamos a crear el archivo de rutas en `envios/urls.py` y escribimos lo siguiente:

Python

```
# envios/urls.py

from django.urls import path
from . import views

urlpatterns = [
    path('', views.dashboard,
    name='dashboard'),
    path('encomiendas/', views.encomienda_lista,
    name='encomienda_lista'),
    path('encomiendas/nueva/', views.encomienda_crear,
    name='encomienda_crear'),
    path('encomiendas/<int:pk>/', views.encomienda_detalle,
    name='encomienda_detalle'),
    path('encomiendas/<int:pk>/estado/', views.encomienda_cambiar_estado,
    name='encomienda_cambiar_estado'),
```

```
]
```

Ahora vamos a registrar las URLs en el enrutador principal `config/urls.py`:

Python

```
# config/urls.py

from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('envios.urls')),
    path('accounts/', include('django.contrib.auth.urls')), # login/logout
    incluidos
]

if settings.DEBUG:
    urlpatterns += static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Django Templates

Las plantillas son la tercera y más importante parte de la estructura MVT de Django. Una plantilla en Django está básicamente escrita en HTML, CSS y Javascript en un archivo .html. El framework Django maneja y genera eficientemente páginas web HTML dinámicas que son visibles para el usuario final. Django funciona principalmente con un backend por lo que, con el fin de proporcionar un frontend y proporcionar un diseño a nuestro sitio web, utilizamos plantillas. Hay dos métodos para añadir la plantilla a nuestro sitio web dependiendo de nuestras necesidades.

Podemos utilizar un único directorio de plantillas que se extenderá por todo el proyecto. Para cada aplicación de nuestro proyecto, podemos crear un directorio de plantillas diferente.

Para nuestro proyecto actual, crearemos un único directorio de plantillas que se extenderá a lo largo de todo el proyecto por simplicidad. Las plantillas a nivel de aplicación se utilizan generalmente en proyectos grandes o en caso de que queramos proporcionar un diseño diferente a cada componente de nuestra página web.

Configurar la estructura de templates

Para ello, vamos a ir al archivo `settings.py` y ubicamos la sección `TEMPLATES`

Python

```
# config/settings.py
```

```
TEMPLATES = [
```

```
{
```

```
    'BACKEND': 'django.template.backends.django.DjangoTemplates',
```

```
    'DIRS': [BASE_DIR / 'templates'], # <- carpeta global de templates
```

```
    'APP_DIRS': True,
```

```
    'OPTIONS': {
```

```
        'context_processors': [
```

```
            'django.template.context_processors.debug',
```

```
            'django.template.context_processors.request',
```

```
            'django.contrib.auth.context_processors.auth',
```

```
        'django.contrib.messages.context_processors.messages',
    ],
},
},
]
```

```
# Archivos estáticos (CSS, JS, imágenes)
```

```
STATIC_URL = 'static/'
```

```
STATICFILES_DIRS = [BASE_DIR / 'static']
```

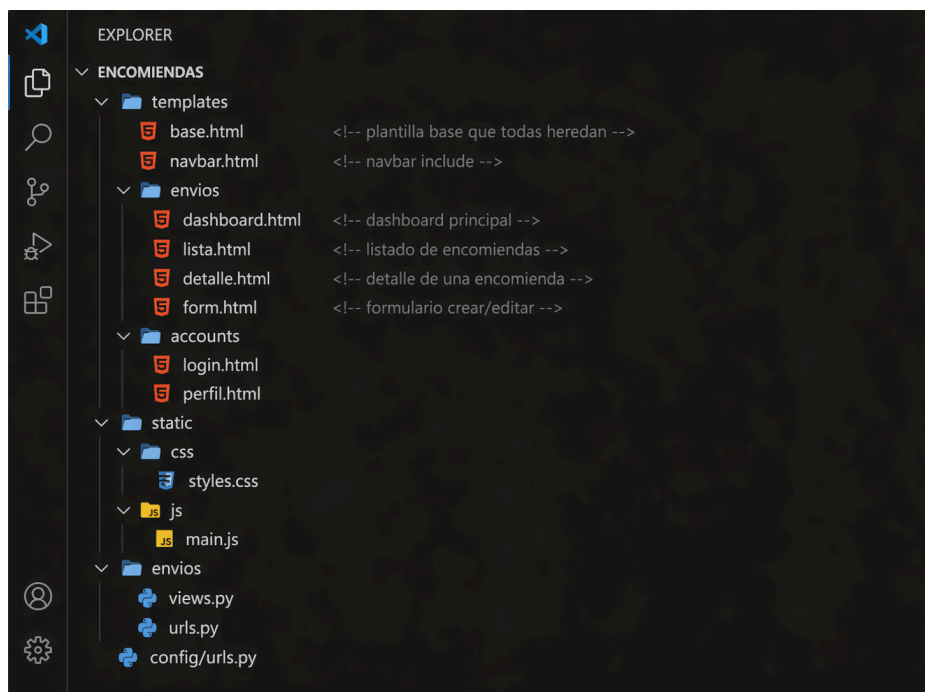
```
STATIC_ROOT = BASE_DIR / 'staticfiles'
```

```
MEDIA_URL = '/media/'
```

```
MEDIA_ROOT = BASE_DIR / 'media'
```

Crear la estructura de directorios

Vamos a crear la siguiente estructura en nuestro proyecto. Además, de las carpetas debemos crear los archivos.



Herencia de Templates

La herencia es el concepto más importante del sistema de templates de Django. Permite que todas las páginas del sistema compartan una estructura común (navbar, footer, CSS) definida una sola vez en una plantilla base. Las páginas hijas solo definen su contenido particular.

VENTAJA CLAVE

Sin herencia: cada página repite el DOCTYPE, head, navbar y footer — cientos de líneas duplicadas.

Con herencia: el navbar y el footer se escriben UNA sola vez en base.html. Cualquier cambio se refleja automáticamente en todas las páginas.

Configurar Plantilla base (base.html)

La plantilla madre define la estructura HTML completa y declara “bloques” (áreas sobrescribibles) con la etiqueta `{% block nombre %}...{% endblock %}`:

Toda página hereda de esta plantilla. Define la estructura HTML común: navbar, mensajes flash, bloque de contenido y footer

Python

```
<!DOCTYPE html>

<html lang="es">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>{% block title %}Encomiendas{% endblock %}</title>

    <!-- Bootstrap CSS -->

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

    <!-- Font Awesome -->

    <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css">

    {% load static %}

    <link rel="stylesheet" href="{% static 'css/styles.css' %}">

    {% block extra_css %}{% endblock %}

</head>

<body>

    {% include 'navbar.html' %}

    <main class="container py-4">

        {% if messages %}

        <div class="messages">

            {% for message in messages %}

                <div class="alert alert-{{ message.tags }} alert-dismissible fade show">
```

```

        {{ message }}

        <button type="button" class="btn-close" data-bs-dismiss="alert"></button>

    </div>

    {% endfor %}

</div>

{% endif %}

{% block content %}{% endblock %}

</main>

<footer class="bg-dark text-white py-3 mt-5">

    <div class="container text-center">

        <p class="mb-0">Sistema de Gestión de Encomiendas &copy; {% now "Y" %}</p>

    </div>

</footer>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
></script>

    <script src="{% static 'js/main.js' %}"></script>

    {% block extra_js %}{% endblock %}

</body>

</html>

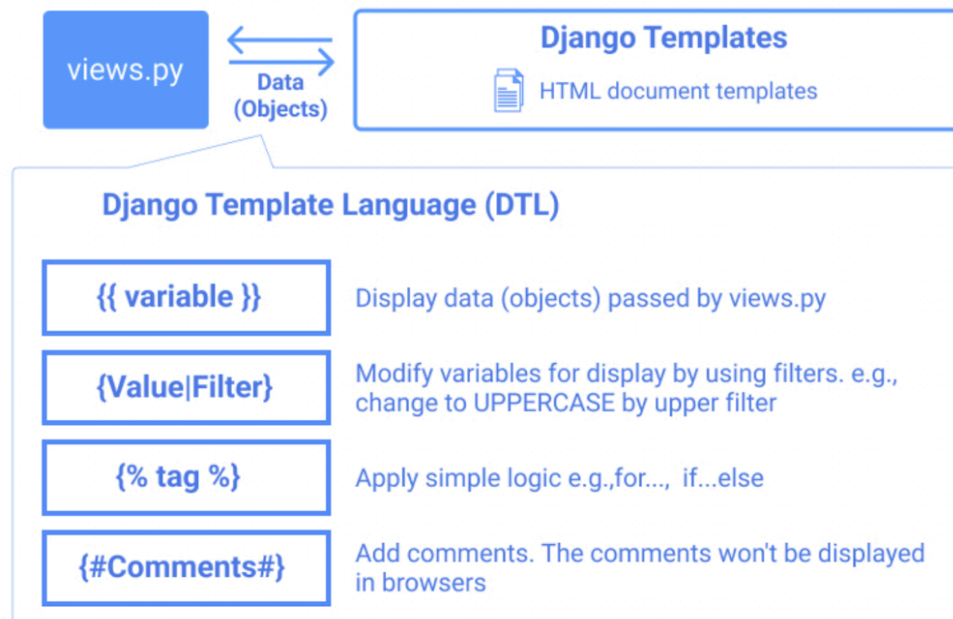
```

Resumen de bloques definidos en base.html

Bloque	Para qué sirve	Obligatorio
title	Título de la pestaña del navegador	Sí
extra_css	CSS adicional que solo carga esta página	No
content	TODO el contenido HTML de la página	Sí
extra_js	JavaScript adicional que solo carga esta página	No

Django Template Language - Etiquetas más usadas

El lenguaje de plantillas de Django (DTL) es un lenguaje basado en texto diseñado para salvar la brecha entre Python y HTML, lo que permite a los desarrolladores generar contenido web dinámico. Está integrado en el marco de trabajo de Django y hace hincapié en una separación clara entre la presentación y la lógica de negocio.



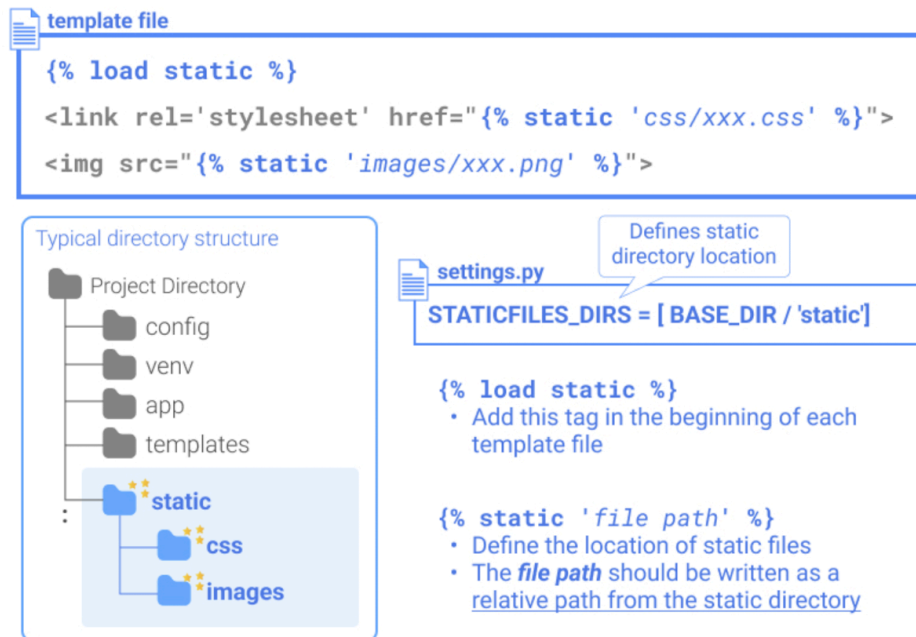
Componentes sintácticos básicos

DTL utiliza cuatro construcciones principales para interactuar con los datos que se pasan desde las vistas:

- Variables: se utilizan para mostrar datos dinámicos. Se escriben entre llaves dobles: `{{ nombre_de_la_variable }}`.
- Etiquetas: controlan la lógica de la plantilla, como bucles y condiciones. Se escriben entre llaves y signos de porcentaje: `{% nombre_de_la_etiqueta %}`.
- Filtros: modifican el resultado de las variables (por ejemplo, cambiando las mayúsculas y minúsculas o formateando fechas). Se aplican utilizando el símbolo de barra vertical: `{{ variable|nombre_del_filtro }}`.
- Comentarios: partes de la plantilla que se ignoran durante la renderización. Utiliza `{# #}` para líneas individuales o `{% comentario %}` para bloques de varias líneas.

Etiqueta	Descripción	Ejemplo
<code>{% load static %}</code>	Carga la librería de archivos estáticos	<code>{% load static %}</code>
<code>{% static 'ruta' %}</code>	Genera la URL de un archivo estático	<code>{% static 'css/styles.css' %}</code>
<code>{% url 'nombre' %}</code>	Genera la URL de una vista por su nombre	<code>{% url 'encomienda_lista' %}</code>
<code>{% include 'file' %}</code>	Incluye otra plantilla dentro	<code>{% include 'navbar.html' %}</code>
<code>{% extends 'base.html' %}</code>	Hereda de la plantilla base	Primera línea de cada template
<code>{% block nombre %}</code>	Define una zona sobrescribible	<code>{% block content %}...{% endblock %}</code>
<code>{{ variable }}</code>	Muestra el valor de una variable	<code>{{ encomienda.codigo }}</code>
<code>{% for x in lista %}</code>	Bucle sobre una lista	<code>{% for enc in encomiendas %}</code>
<code>{% if condicion %}</code>	Condicional	<code>{% if enc.tiene_retraso %}</code>
<code>{% csrf_token %}</code>	Token de seguridad para formularios POST	Dentro de todo <code><form></code>

`{% static %}` tag



En tus plantillas Django, utiliza la etiqueta de plantilla `{% static %}` para enlazar con tus archivos estáticos. Comienza cargando la biblioteca de etiquetas de plantillas estáticas en la parte superior de tu archivo de plantilla:

Python

```
{% load static %}
```

{% include <file.html> %} tag

La etiqueta include le permite incluir una plantilla dentro de la plantilla actual. Esto es útil cuando se tiene un bloque de contenido que es el mismo para muchas páginas.

Python

```
{% include 'navbar.html' %}
```

Template variables

En las plantillas de Django, puedes renderizar variables poniéndolas dentro de corchetes `{{ }}`:

```

<!-- Contenido principal -->
<main class="container py-4">
    {% if messages %}
    <div class="messages">
        {% for message in messages %}
        <div class="alert alert-{{ message.tags }} alert-dismissible fade show">
            {{ message }}
            <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
        </div>
        {% endfor %}
    </div>
    {% endif %}

    {% block content %}{% endblock %}
</main>

```

Template tags

En las plantillas de Django, puedes realizar lógica de programación como ejecutar sentencias if y bucles for.

Estas palabras clave, if y for, se llaman "etiquetas de plantilla" en Django.

Para ejecutar etiquetas de plantilla, las encerramos entre corchetes {% %}.

```

<!-- Contenido principal -->
<main class="container py-4">
    {% if messages %}
    <div class="messages">
        {% for message in messages %}
        <div class="alert alert-{{ message.tags }} alert-dismissible fade show">
            {{ message }}
            <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
        </div>
        {% endfor %}
    </div>
    {% endif %}

    {% block content %}{% endblock %}
</main>

```

Características principales

- Herencia de plantillas: esta es la función más potente de DTL, ya que permite crear un «esqueleto» base.html con elementos comunes (como barras de navegación) que otras páginas pueden ampliar.
- Escapado automático: para prevenir ataques de tipo Cross-Site Scripting (XSS), Django escapa automáticamente caracteres HTML como <, > y &.
- Sin código arbitrario: por diseño, DTL no puede ejecutar expresiones Python arbitrarias. Esto garantiza que los diseñadores puedan trabajar en plantillas sin necesidad de tener profundos conocimientos de programación.
- Extensibilidad: si las herramientas integradas no son suficientes, puedes crear etiquetas y filtros personalizados utilizando Python.

Vistas basadas en funciones y clases

Las vistas basadas en funciones (FBV) ofrecen simplicidad y control explícito, siendo ideales para lógica sencilla. Las vistas basadas en clases (CBV) mejoran la organización, reutilización y extensibilidad mediante herencia, ideales para proyectos grandes o tareas estándar. Ambas son válidas y pueden usarse conjuntamente en Django. Para este proyecto usaremos principalmente vistas basadas en funciones por su claridad y facilidad de aprendizaje.

Vistas basadas en funciones

Como su nombre indica, las vistas basadas en funciones se implementan como funciones de Python. Para entender cómo funcionan, considere el siguiente fragmento, que muestra una función de vista simple llamada `home_page`:

```
from django.http import HttpResponse
def home_page(request):
    message = "<html><h1>Welcome to my Website</h1></html>"
    return HttpResponse(message)
```

La función de vista definida aquí, llamada `home_page`, toma un objeto `request` como argumento y devuelve un objeto `HttpResponse` con un mensaje `Welcome to my Website`. La ventaja de



utilizar vistas basadas en funciones es que, al estar implementadas como simples funciones de Python, son más fáciles de aprender y también de leer para otros programadores. La mayor desventaja de las vistas basadas en funciones es que el código no puede reutilizarse y hacerse tan conciso como las vistas basadas en clases para casos de uso genérico. La siguiente sección es una breve introducción a las vistas basadas en clases.

Vistas basadas en clases

Como su nombre indica, las vistas basadas en clases se implementan como clases de Python. Utilizando los principios de la herencia de clases, estas clases se implementan como subclases de las clases de vistas genéricas de Django. A diferencia de las vistas basadas en funciones, donde toda la lógica de la vista se expresa explícitamente en una función, las clases de vistas genéricas de Django vienen con varias propiedades y métodos pre-construidos que pueden proporcionar atajos para escribir vistas limpias y reutilizables. Esta propiedad resulta útil con bastante frecuencia durante el desarrollo web; por ejemplo, los desarrolladores a menudo necesitan renderizar una página HTML sin necesidad de insertar ningún dato de la base de datos, o cualquier personalización específica para el usuario. En este caso, es posible simplemente heredar de `TemplateView` de Django, y especificar la ruta del archivo HTML. El siguiente es un ejemplo de una vista basada en clases que puede mostrar el mismo mensaje que en el ejemplo de vista basada en funciones:

```
from django.views.generic import TemplateView
class HomePage(TemplateView):
    template_name = 'home_page.html'
```

En el fragmento de código anterior, `HomePage` es una vista basada en clases que hereda `TemplateView` de Django del módulo `django.views.generic`. El atributo de clase `template_name` define la plantilla a mostrar cuando la vista es invocada.

Herencia de Templates

Todo template hijo comienza con `{% extends 'base.html' %}` como primera línea, y luego define sólo los bloques que necesita sobrescribir.

Dashboard del Sistema

El dashboard muestra las estadísticas en tiempo real del sistema. Hereda de `base.html` y usa variables del contexto que la vista envió:

Python

```
{% extends 'base.html' %}

{% block title %}Dashboard - Encomiendas{% endblock %}

{% block content %}

<div class="row mb-4">

    <div class="col-12">

        <h2 class="h3"><i class="fas fa-tachometer-alt me-2"></i>Dashboard</h2>

    </div>

</div>


<!-- Tarjetas de estadísticas -->

<div class="row mb-4">

    {% for label, valor, color, icono in stats %}

    <div class="col-md-3">

        <div class="card bg-{{ color }} text-white shadow-sm mb-3">

            <div class="card-body d-flex justify-content-between align-items-center">

                <div>

                    <h6 class="card-title">{{ label }}</h6>

                    <h2 class="mb-0">{{ valor }}</h2>

                </div>

                <i class="fas fa-{{ icono }} fa-2x"></i>

            </div>

        </div>

    </div>

</div>
```

```

</div>

{% endfor %}
</div>

<!-- Últimas encomiendas -->
<div class="card shadow">
  <div class="card-header bg-white">
    <h5 class="mb-0">Últimas encomiendas registradas</h5>
  </div>
  <div class="card-body p-0">
    <table class="table table-hover mb-0">
      <thead>
        <tr>
          <th>Código</th><th>Remitente</th>
          <th>Destino</th><th>Estado</th><th></th>
        </tr>
      </thead>
      <tbody>
        {% for enc in ultimas %}
        <tr>
          <td><code>{{ enc.codigo }}</code></td>
          <td>{{ enc.remitente.nombre_completo }}</td>
          <td>{{ enc.ruta.destino }}</td>
          <td>
            <span class="badge bg-{{ if enc.esta_en_transito %}}primary

```

```

        {% elif enc.esta_entregada %}success

        {% elif enc.tiene_retraso %}danger

        {% else %}secondary{% endif %}">

        {{ enc.get_estado_display }}

    </span>

</td>

    <td><a href="{% url 'encomienda_detalle' enc.pk %}" class="btn btn-sm
btn-outline-primary">Ver</a></td>

</tr>

{% empty %}

<tr><td colspan="5" class="text-center py-3">No hay encomiendas</td></tr>

{% endfor %}

</tbody>

</table>

</div>

</div>

{% endblock %}

```

Listado con paginación y filtros

HTML

```

{# templates/envios/lista.html #}

{% extends 'base.html' %}

```

```

{% block title %}Encomiendas{% endblock %}

{% block content %}
<div class="d-flex justify-content-between align-items-center mb-4">
    <h2 class="h3">
        <i class="fas fa-boxes me-2"></i>Encomiendas
    </h2>
    <a href="{% url 'encomienda_crear' %}" class="btn btn-primary">
        <i class="fas fa-plus me-1"></i>Nueva
    </a>
</div>

{# Formulario de búsqueda y filtros (metodo GET) #}
<form method="get" class="row g-2 mb-3">
    <div class="col-md-6">
        <input type="text" name="q" value="{{ request.GET.q }}"
            class="form-control" placeholder="Buscar por código o cliente...">
    </div>
    <div class="col-md-4">
        <select name="estado" class="form-select">
            <option value="">Todos los estados</option>
            {% for valor, etiqueta in estados %}
            <option value="{{ valor }}"
                {% if estado_activo == valor %}selected{% endif %}>
                {{ etiqueta }}
            </option>
            {% endfor %}
        </select>
    </div>
</form>

```

```

        </select>
    </div>
    <div class="col-md-2">
        <button type="submit" class="btn btn-outline-primary w-100">Filtrar</button>
    </div>
</form>

{# Tabla de encomiendas #}
<div class="card shadow">
    <div class="card-body p-0">
        <table class="table table-hover mb-0">
            <thead>
                <tr>
                    <th>Código</th><th>Remitente</th>
                    <th>Ruta</th><th>Estado</th><th>Fecha</th><th></th>
                </tr>
            </thead>
            <tbody>
                {% for enc in encomiendas %}
                <tr>
                    <td><code>{{ enc.codigo }}</code></td>
                    <td>{{ enc.remitente.nombre_completo }}</td>
                    <td>{{ enc.ruta.origen }} → {{ enc.ruta.destino }}</td>
                    <td>
                        <span class="badge-estado badge-{{ enc.estado }}">
                            {{ enc.get_estado_display }}
                        </span>
                    </td>
                </tr>
            </tbody>
        </table>
    </div>
</div>

```

```

        <td>{{ enc.fecha_registro|date:'d/m/Y' }}</td>
        <td>
            <a href="{% url 'encomienda_detalle' enc.pk %}"
                class="btn btn-sm btn-outline-primary">Ver</a>
        </td>
    </tr>
    {% empty %}
    <tr><td colspan="6" class="text-center py-4">
        No se encontraron encomiendas
    </td></tr>
    {% endfor %}
</tbody>
</table>
</div>

{# Paginacion #}
{% if encomiendas.has_other_pages %}
<div class="card-footer bg-white">
    <nav><ul class="pagination justify-content-center mb-0">
        {% if encomiendas.has_previous %}
        <li class="page-item">
            <a class="page-link" href="?page={{ encomiendas.previous_page_number
            }}&q={{ request.GET.q }}&estado={{ request.GET.estado }}">
                &laquo; Anterior
            </a>
        </li>
        {% endif %}
        {% for num in encomiendas.paginator.page_range %}
        <li class="page-item {% if encomiendas.number == num %}active{% endif %}">

```

```

        <a class="page-link" href="?page={{ num }}&q={{ request.GET.q }}&estado={{
request.GET.estado }}">{{ num }}</a>

    </li>

    {% endfor %}

    {% if encomiendas.has_next %}

    <li class="page-item">

        <a class="page-link" href="?page={{ encomiendas.next_page_number }}&q={{
request.GET.q }}&estado={{ request.GET.estado }}">

            Siguiente &raquo;

        </a>

    </li>

    {% endif %}

</ul></nav>

</div>

{% endif %}

</div>

{% endblock %}

```

Detalle de encomienda con historial

HTML

```

{%# templates/envios/detalle.html #}

{% extends 'base.html' %}

{% block title %}{{ encomienda.codigo }} - Detalle{% endblock %}

```



```

{% block content %}
<div class="d-flex justify-content-between align-items-center mb-4">
  <h2 class="h3">
    <i class="fas fa-box me-2"></i>{{ encomienda.codigo }}
  </h2>
  <div>
    <a href="{% url 'encomienda_lista' %}" class="btn btn-outline-secondary me-2">
      Volver
    </a>
    {# Solo mostrar si no está entregada ni devuelta #}
    {% if not encomienda.esta_entregada %}
      <button class="btn btn-warning" data-bs-toggle="modal"
data-bs-target="#modalEstado">
        Cambiar estado
      </button>
    {% endif %}
  </div>
</div>

<div class="row">
  {# Columna izquierda: datos principales #}
  <div class="col-md-8">
    <div class="card shadow mb-4">
      <div class="card-header bg-white d-flex justify-content-between">
        <h5 class="mb-0">Información de la encomienda</h5>
        <span class="badge-estado badge-{{ encomienda.estado }}">
          {{ encomienda.get_estado_display }}
        </span>
      </div>
    </div>
  </div>
</div>

```

```

<div class="card-body">
  <div class="row mb-3">
    <div class="col-md-6">
      <p class="text-muted mb-1">Remitente</p>
      <h6>{{ encomienda.remitente.nombre_completo }}</h6>
    </div>
    <div class="col-md-6">
      <p class="text-muted mb-1">Destinatario</p>
      <h6>{{ encomienda.destinatario.nombre_completo }}</h6>
    </div>
  </div>
  <div class="row">
    <div class="col-md-4">
      <p class="text-muted mb-1">Ruta</p>
      <h6>{{ encomienda.ruta }}</h6>
    </div>
    <div class="col-md-4">
      <p class="text-muted mb-1">Peso</p>
      <h6>{{ encomienda.peso_kg }} kg</h6>
    </div>
    <div class="col-md-4">
      <p class="text-muted mb-1">Costo</p>
      <h6>S/ {{ encomienda.costo_envio|floatformat:2 }}</h6>
    </div>
  </div>

  {# Alerta de retraso usando @property del modelo #}
  {% if encomienda.tiene_retraso %}

```

```

<div class="alert alert-danger mt-3">
    <i class="fas fa-exclamation-triangle me-2"></i>
    Esta encomienda tiene {{ encomienda.dias_en_transito }} días en tránsito
    y superó la fecha de entrega estimada.
</div>

{% endif %}
</div>
</div>

{# Historial de estados #}
<div class="card shadow">
    <div class="card-header bg-white">
        <h5 class="mb-0">Historial de estados</h5>
    </div>
    <div class="card-body p-0">
        <table class="table mb-0">
            <thead>

<tr><th>Fecha</th><th>Anterior</th><th>Nuevo</th><th>Empleado</th><th>Observación<
/th></tr>

            </thead>
            <tbody>
                {% for h in historial %}
                <tr>
                    <td>{{ h.fecha_cambio|date:'d/m/Y H:i' }}</td>
                    <td><span class="badge-estado badge-{{ h.estado_anterior }}">{{
h.get_estado_anterior_display }}</span></td>
                    <td><span class="badge-estado badge-{{ h.estado_nuevo }}">{{
h.get_estado_nuevo_display }}</span></td>
                    <td>{{ h.empleado }}</td>

```

```

        <td>{{ h.observacion|default:'-' }}</td>
    </tr>
    {% empty %}
    <tr><td colspan="5">Sin cambios registrados</td></tr>
    {% endfor %}
</tbody>
</table>
</div>
</div>
</div>
</div>
{% endblock %}

{# JS adicional: solo carga en esta pagina #}
{% block extra_js %}
<script>
    {# Auto-seleccionar el estado actual en el modal #}
    document.getElementById('selectEstado').value = '{{ encomienda.estado }}';
</script>
{% endblock %}

```

Archivos Estáticos

Django NO sirve los archivos estáticos directamente. La etiqueta `{% static %}` convierte una ruta relativa en la URL correcta según la configuración `STATIC_URL` de `settings.py`.

Configuración en [settings.py](#)

Python

```
# config/settings.py

# URL base para los archivos estaticos
STATIC_URL = 'static/'

# Carpetas donde Django busca los archivos en DESARROLLO
STATICFILES_DIRS = [
    BASE_DIR / 'static',    # <- aqui pones tus CSS, JS, imagenes
]

# Carpeta donde se juntan todos al hacer collectstatic (PRODUCCION)
STATIC_ROOT = BASE_DIR / 'staticfiles'

# Archivos subidos por el usuario (formularios con FileField/ImageField)
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

Estructura de carpetas

encomiendas/	
static/	<- STATICFILES_DIRS apunta aquí
css/	
styles.css	<- {% static 'css/styles.css' %}
js/	
main.js	<- {% static 'js/main.js' %}
img/	
logo.png	<- {% static 'img/logo.png' %}
> staticfiles/	<- STATIC_ROOT (generado por collectstatic)
> templates/	
manage.py	

Uso correcto en los templates

Python

```
{# SIEMPRE cargar static antes del primer uso #}
{% load static %}

{# — En el <head> ————— #}

{# Bootstrap desde CDN: no necesita {% static %} #}
<link href="https://cdn.../bootstrap.min.css" rel="stylesheet">

{# Tu CSS propio: SI necesita {% static %} #}
<link rel="stylesheet" href="{% static 'css/styles.css' %}">

{# — En el <body> ————— #}

{# Imagen: ruta estatica #}
```

```


{# JS propio al final del body #}
<script src="{% static 'js/main.js' %}"></script>

{# — Resultado generado en el HTML ————— #}
{# {% static 'css/styles.css' %} -> /static/css/styles.css #}
{# {% static 'js/main.js' %}      -> /static/js/main.js      #}
{# {% static 'img/logo.png' %}   -> /static/img/logo.png   #}
```

Servir estáticos en desarrollo con Docker

Python

```
# config/urls.py

from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('envios.urls')),
    path('accounts/', include('django.contrib.auth.urls')),
]

# En DEBUG=True, Django sirve los archivos estaticos automaticamente
```

```
if settings.DEBUG:
    urlpatterns += static(settings.STATIC_URL,
                           document_root=settings.STATIC_ROOT)

    urlpatterns += static(settings.MEDIA_URL,
                           document_root=settings.MEDIA_ROOT)
```

COLLECTSTATIC EN DOCKER

En produccion debes juntar todos los archivos estaticos en STATIC_ROOT:

```
docker compose exec web python manage.py collectstatic
```

Esto copia todo de STATICFILES_DIRS a STATIC_ROOT (la carpeta staticfiles/).
En desarrollo con DEBUG=True esto NO es necesario.

CSS Personalizado — styles.css completo

El archivo `static/css/styles.css` se carga en todas las páginas a través de `base.html` y complementa los estilos de Bootstrap con el diseño propio del sistema de encomiendas.

Variables de color y estructura base

static/css/styles.css — parte 1/4

Python

```
# config/urls.py

from django.contrib import admin
from django.urls import path, include
```



```

/* static/css/styles.css */

/* — Variables globales del sistema ————— */
:root {
  /* Colores de estados de encomienda */
  --estado-pendiente: #6c757d; /* gris */
  --estado-transito: #0d6efd; /* azul */
  --estado-destino: #fd7e14; /* naranja */
  --estado-entregado: #198754; /* verde */
  --estado-devuelto: #dc3545; /* rojo */
  /* Colores generales del sistema */
  --color-primary: #0d6efd;
  --color-bg: #f8f9fa;
  --shadow-sm: 0 2px 8px rgba(0,0,0,.07);
}

/* — Cuerpo general ————— */
body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: var(--color-bg);
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

main { flex: 1; } /* el footer siempre queda abajo */

```

Navbar y tarjetas del dashboard

static/css/styles.css — parte 2/4

Python

```
/* — Navbar ————— */

.navbar-brand {
  font-weight: 600;
  letter-spacing: .3px;
}

.navbar .nav-link {
  font-weight: 500;
  transition: opacity .15s;
}

.navbar .nav-link:hover { opacity: .85; }

/* Link de la pagina actual */
.navbar .nav-link.active {
  font-weight: 600;
  border-bottom: 2px solid rgba(255,255,255,.6);
}

/* — Cards generales ————— */
.card {
  border: none;
  border-radius: .5rem;
  box-shadow: var(--shadow-sm);
```

```

    transition: transform .15s ease, box-shadow .15s ease;
}

.card:hover {
    transform: translateY(-2px);
    box-shadow: 0 4px 14px rgba(0,0,0,.1);
}

.card-header {
    font-weight: 600;
    background-color: #fff;
    border-bottom: 1px solid rgba(0,0,0,.06);
}

/* — Tarjetas de estadísticas del dashboard ————— */
.stat-card { border-left: 4px solid; }
.stat-card.activas { border-color: var(--estado-transito); }
.stat-card.transito { border-color: var(--estado-transito); }
.stat-card.retraso { border-color: var(--estado-devuelto); }
.stat-card.entregadas { border-color: var(--estado-entregado); }

```

Badges de estado de encomienda

El sistema usa clases dinámicas basadas en el código de estado (PE, TR, DE, EN, DV). El template genera `class="badge-estado badge-{{ enc.estado }}"` y el CSS aplica el color correcto automáticamente.

static/css/styles.css — parte 3/4

Python

```
/* — Badges de estado de encomienda ————— */
/*
 * Uso en el template:
 * <span class="badge-estado badge-{{ enc.estado }}">
 *   {{ enc.get_estado_display }}
 * </span>
 *
 * Django inyecta 'PE', 'TR', 'DE', 'EN' o 'DV' en lugar de {{ enc.estado }}
 * El CSS aplica el color correcto automaticamente.
 */

.badge-estado {
    display: inline-block;
    padding: 3px 10px;
    border-radius: 12px;
    font-size: .78rem;
    font-weight: 600;
    letter-spacing: .3px;
    white-space: nowrap;
}

.badge-PE {                                /* Pendiente */
    background-color: #e9ecef;
    color: #495057;
}

.badge-TR {                                /* En transito */
    background-color: #cfe2ff;
```

```

    color: #084298;
}
.badge-DE {                                /* En destino */
    background-color: #fff3cd;
    color: #664d03;
}
.badge-EN {                                /* Entregado */
    background-color: #d1e7dd;
    color: #0a3622;
}
.badge-DV {                                /* Devuelto */
    background-color: #f8d7da;
    color: #58151c;
}

```

Tablas, formularios y animaciones

static/css/styles.css — parte 3/4

Python

```

/* — Tablas ————— */
.table thead th {
    background-color: #f8f9fa;
    font-weight: 600;
    font-size: .82rem;
    text-transform: uppercase;
    letter-spacing: .5px;
}

```

```
color: #6c757d;
border-bottom: 2px solid #dee2e6;
}

.table tbody tr:hover {
  background-color: rgba(13,110,253,.04);
  cursor: pointer;
}

/* — Formularios ————— */
.form-control:focus,
.form-select:focus {
  border-color: #86b7fe;
  box-shadow: 0 0 0 .25rem rgba(13,110,253,.15);
}

/* — Paginacion ————— */
.pagination .page-link { color: var(--color-primary); }

.pagination .page-item.active .page-link {
  background-color: var(--color-primary);
  border-color: var(--color-primary);
}

/* — Alertas flash: desaparecen automaticamente ————— */
.alert {
  animation: fadeAlert 5s ease forwards;
}
```

```

@keyframes fadeAlert {
  0% { opacity: 1; max-height: 200px; }
  70% { opacity: 1; }
  100%{ opacity: 0; max-height: 0; padding: 0; margin: 0; overflow: hidden; }
}

/* — Sidebar del dashboard (columna lateral) ————— */
.sidebar-link {
  display: block;
  padding: .5rem 1rem;
  color: #495057;
  text-decoration: none;
  border-radius: .375rem;
  transition: background .12s;
}

.sidebar-link:hover { background: rgba(13,110,253,.08); color: #0d6efd; }
.sidebar-link.active { background: #e7f1ff; color: #0d6efd; font-weight: 600; }

```

JavaScript básico — main.js

El archivo `static/js/main.js` contiene funciones reutilizables que se ejecutan en todas las páginas. Se carga al final del body en `base.html`.

Python

```
// static/js/main.js

document.addEventListener('DOMContentLoaded', function () {

    // — Inicializar tooltips de Bootstrap —————
    const tooltips = document.querySelectorAll('[data-bs-toggle="tooltip"]');
    tooltips.forEach(el => new bootstrap.Tooltip(el));

    // — Auto-cerrar alertas flash despues de 5 segundos ———
    // (complementa la animacion CSS del styles.css)
    setTimeout(function () {
        document.querySelectorAll('.alert').forEach(function (alert) {
            const bsAlert = bootstrap.Alert.getOrCreateInstance(alert);
            bsAlert.close();
        });
    }, 5000);

    // — Confirmacion antes de eliminar —————
    // Uso en el template:
    // <button onclick="return confirmar('Eliminar este registro?')>
    //      form="formEliminar">Eliminar</button>
    window.confirmar = function (mensaje) {
        return confirm(mensaje || '¿Estás seguro?');
    };

});
```



```
// — Resaltar fila al hacer clic (navegacion intuitiva) —
// Uso: <tr class="fila-link" data-href="{% url 'encomienda_detalle' enc.pk %}">
document.querySelectorAll('.fila-link').forEach(function (fila) {
  fila.addEventListener('click', function () {
    window.location = this.dataset.href;
  });
});
```

USO DEL JS ADICIONAL POR PÁGINA

Para cargar JS solo en una pagina específica usa el bloque extra_js:

```
{% block extra_js %}
<script>
  // Este codigo solo carga en esta pagina
  document.getElementById('selectEstado').value = '{{ encomienda.estado }}';
</script>
{% endblock %}
```

Include — Reutilizar fragmentos de template

La etiqueta `{% include %}` inserta otra plantilla dentro de la actual. Ideal para elementos que se repiten en varias páginas como paginación, tarjetas de estadística o el navbar.

Partial: tarjeta de estadística reutilizable

Python

```
{# templates/partials/stat_card.html #}
<div class="col-md-3">
```

```

<div class="card stat-card shadow-sm mb-3" style="border-left-color: {{
color_hex }}">

    <div class="card-body d-flex justify-content-between align-items-center">

        <div>

            <h6 class="card-title text-muted mb-1">{{ label }}</h6>
            <h2 class="mb-0">{{ valor }}</h2>

        </div>

        <i class="fas fa-{{ icono }} fa-2x text-muted opacity-50"></i>

    </div>

</div>

```

Uso en el dashboard con {% include %} y paso de variables:

Python

```

{# templates/envios/dashboard.html #}

<div class="row mb-4">

    {% include 'partials/stat_card.html' with
        label='Activas' valor=total_activas
        icono='shipping-fast' color_hex='#0d6efd' %}

    {% include 'partials/stat_card.html' with
        label='En tránsito' valor=en_transito
        icono='truck' color_hex='#0d6efd' %}

    {% include 'partials/stat_card.html' with
        label='Con retraso' valor=con_retraso
        icono='exclamation-triangle' color_hex='#dc3545' %}


```

```

{% include 'partials/stat_card.html' with
    label='Entregadas hoy' valor=entregadas_hoy
    icono='check-circle' color_hex='#198754' %}
</div>

```

Partial: paginación reutilizable

Python

```

{# templates/partials/paginacion.html #}

{% if page_obj.has_other_pages %}
<nav aria-label="Paginación">
    <ul class="pagination justify-content-center mb-0">

        {# Boton anterior #}
        {% if page_obj.has_previous %}
        <li class="page-item">
            <a class="page-link" href="?page={{ page_obj.previous_page_number }}">
                &laquo;
            </a>
        </li>
        {% else %}
        <li class="page-item disabled">
            <span class="page-link">&laquo;</span>
        </li>

```

```

{% endif %}

{# Numeros de pagina #}
{% for num in page_obj.paginator.page_range %}
<li class="page-item {% if page_obj.number == num %}active{% endif %}">
<a class="page-link" href="?page={{ num }}">{{ num }}</a>
</li>
{% endfor %}

{# Boton siguiente #}
{% if page_obj.has_next %}
<li class="page-item">
<a class="page-link" href="?page={{ page_obj.next_page_number }}">
&raquo;
</a>
</li>
{% else %}
<li class="page-item disabled">
<span class="page-link">&raquo;</span>
</li>
{% endif %}

</ul>
</nav>
{% endif %}

```

Uso en cualquier lista con paginación:

Python

```
{# templates/envios/lista.html #}

{% block content %}

    ...

    <div class="card-footer bg-white">

        {% include 'partials/paginacion.html' with page_obj=encomiendas %}

    </div>

{% endblock %}
```

Filtros de template más usados

Los filtros de plantillas (templates) en Django son funcionalidades integradas o personalizadas que permiten modificar el formato, la presentación o el valor de las variables directamente en el HTML antes de renderizarlo. Se aplican usando el símbolo de tubería (|) dentro de las etiquetas de variable `{{ variable|filtro }}`, facilitando tareas como formatear fechas, convertir texto a mayúsculas, truncar cadenas o establecer valores predeterminados.

Los filtros transforman el valor de una variable antes de mostrarlo. Se aplican con el símbolo | después de la variable.

Características clave de los filtros en Django:

- **Sintaxis:** Se utilizan después de la variable con el símbolo |, por ejemplo, `{{ nombrellower }}` convierte el texto a minúsculas.
- **Encadenamiento:** Se pueden aplicar múltiples filtros secuencialmente, donde el resultado de uno pasa al siguiente (ej. `{{ textolsafellower }}`).
- **Argumentos:** Algunos filtros aceptan parámetros adicionales para refinar su acción, usando dos puntos (ej. `{{ listalslice:"3" }}`).
- **Filtros comunes integrados:**
 - `default`: Proporciona un valor predeterminado si la variable está vacía o es falsa.
 - `length`: Devuelve el número de elementos de una lista o longitud de una cadena.
 - `lower/upper`: Convierte texto a minúsculas o mayúsculas.
 - `capfirst`: Capitaliza la primera letra.
 - `date`: Formatea objetos de fecha/tiempo.

- Personalización: Django permite crear filtros propios si los integrados no cubren las necesidades específicas del proyecto

Filtro	Ejemplo	Resultado
date	<code>{{ enc.fecha_registro date:'d/m/Y' }}</code>	18/04/2026
date con hora	<code>{{ enc.fecha_registro date:'d/m/Y H:i' }}</code>	18/04/2026 10:30
truncatechars	<code>{{ enc.descripcion truncatechars:50 }}</code>	Caja con ropa y docume... (50 chars)
default	<code>{{ enc.observaciones default:'—' }}</code>	— (si el campo es vacío o None)
floatformat	<code>{{ enc.costo_envio floatformat:2 }}</code>	25.00
floatformat	<code>{{ enc.peso_kg floatformat:1 }}</code>	3.5
length	<code>{{ enc.historial.all length }}</code>	Número de registros en el historial
lower	<code>{{ enc.codigo lower }}</code>	enc-2026-001
upper	<code>{{ enc.codigo upper }}</code>	ENC-2026-001
join	<code>{{ lista_estados join:', ' }}</code>	Pendiente, En tránsito, Entregado
yesno	<code>{{ enc.esta_entregada yesno:'Si,No,—' }}</code>	Si (True) / No (False) / — (None)
safe	<code>{{ html_content safe }}</code>	Renderiza HTML sin escapar (cuidado)

ENTREGABLE — TEMPLATES, HERENCIA Y ESTÁTICOS

Al finalizar este complemento debes poder demostrar:

Herencia

1. base.html con los 4 bloques: title, extra_css, content, extra_js.
2. dashboard.html hereda de base.html y define su block content.
3. lista.html hereda de base.html con filtros GET para búsqueda y estado.
4. detalle.html muestra los @property del modelo (tiene_retrazo, dias_en_transito).

Archivos estaticos

5. Carpeta static/ con css/styles.css y js/main.js creados.
6. {% load static %} en base.html antes del primer {% static %}.
7. urls.py sirve los estaticos con if settings.DEBUG.

CSS

8. Variables CSS :root con los colores de cada estado de encomienda.
9. Clases .badge-PE, .badge-TR, .badge-DE, .badge-EN, .badge-DV con colores distintos.
10. Animacion fadeAlert para auto-cerrar los mensajes flash.

Includes

11. partial stat_card.html reutilizado con {% include with variable=valor %}.
12. partial paginacion.html reutilizado en lista.html.

Vistas en Django

Una vista en Django es una función o clase Python que recibe un objeto `HttpRequest` y devuelve un objeto `HttpResponse`. Es el puente entre el modelo (datos) y el template (presentación). Toda la lógica de negocio que no esté en el modelo vive aquí.

- Recibe la solicitud HTTP del usuario
- Consulta los modelos a través del ORM
- Prepara el contexto (datos para el template)
- Devuelve una respuesta (HTML, JSON, redirect, etc.)

Tipo	Cuándo usar	Ejemplo en el proyecto
FBV (función)	Lógica personalizada, control explícito	<code>dashboard()</code> , <code>encomienda_crear()</code>
CBV (clase)	Operaciones CRUD estándar, reutilización	<code>EncomiendaListView</code> , <code>EncomiendaDetailView</code>
Vista genérica	CRUD completo con mínimo código	<code>ListView</code> , <code>DetailView</code> , <code>CreateView</code>

Vistas Basadas en Funciones (FBV)

Las FBV son funciones Python simples. Reciben un objeto `request` como primer argumento y devuelven un objeto `response`. Son más fáciles de leer y aprender.

Estructura mínima y vista real

Python

```
# envios/views.py

from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth.decorators import login_required
```



```

from django.contrib import messages
from django.utils import timezone

from .models import Encomienda, Empleado, HistorialEstado
from clientes.models import Cliente
from rutas.models import Ruta
from config.choices import EstadoEnvio

# — Vista mínima —————
def mi_vista(request):
    # 1. Recibe el request
    # 2. Ejecuta lógica
    # 3. Devuelve una respuesta
    from django.http import HttpResponse
    return HttpResponse('Hola desde Django')

# — Vista real: dashboard del sistema —————
@login_required
def dashboard(request):
    """Vista principal del sistema con estadísticas"""
    hoy = timezone.now().date()
    context = {
        'total_activas': Encomienda.objects.activas().count(),
        'en_transito': Encomienda.objects.en_transito().count(),
        'con_retraso': Encomienda.objects.con_retraso().count(),
        'entregadas_hoy': Encomienda.objects.filter(

```

```
        estado=EstadoEnvio.ENTREGADO,
        fecha_entrega_real=hoy).count(),
'ultimas':      Encomienda.objects.con_relaciones()[:5],
}
return render(request, 'envios/dashboard.html', context)
```

Las cuatro funciones de atajo más usadas

Python

render() - renderizar un template con contexto

```
from django.shortcuts import render
```

```
def encomienda_lista(request):
    encomiendas = Encomienda.objects.all()
    return render(request, 'envios/lista.html', {
        'encomiendas': encomiendas,
    })
```

redirect() - redirigir a otra URL

```
from django.shortcuts import redirect
```

```
def mi_vista(request):
    return redirect('encomienda_lista')          # por nombre
    return redirect('encomienda_detalle', pk=1)  # con argumento
```

```

    return redirect('/encomiendas/') # por URL directa

# get_object_or_404() - buscar o devolver 404
from django.shortcuts import get_object_or_404

def encomienda_detalle(request, pk):
    # Si no existe el pk → devuelve 404 automáticamente
    # Nunca más: try/except Encomienda.DoesNotExist
    enc = get_object_or_404(Encomienda, pk=pk)
    # También acepta QuerySets optimizados:
    enc = get_object_or_404(Encomienda.objects.con_relaciones(), pk=pk)
    return render(request, 'envios/detalle.html', {'encomienda': enc})

# get_list_or_404() - lista o devolver 404
from django.shortcuts import get_list_or_404

def encomiendas_por_ruta(request, ruta_pk):
    # Si la lista está vacía → devuelve 404
    encomiendas = get_list_or_404(Encomienda, ruta__pk=ruta_pk)
    return render(request, 'envios/lista.html', {
        'encomiendas': encomiendas,
    })

```

Patrón GET/POST — vista de creación completa

Todas las vistas con formulario siguen este patrón: GET muestra el formulario, POST lo procesa y redirige (patrón Post/Redirect/Get):

Python

```
# envios/views.py

@login_required
def encomienda_crear(request):
    """
    GET → muestra el formulario vacío
    POST → valida, guarda y redirige al detalle
    """

    from .forms import EncomiendaForm

    if request.method == 'POST':
        form = EncomiendaForm(request.POST)
        if form.is_valid():
            enc = form.save(commit=False) # no guarda aún en BD
            enc.empleado_registro = Empleado.objects.get(
                email=request.user.email
            )
            enc.save() # ahora sí guarda
            messages.success(
                request,
                f'Encomienda {enc.codigo} registrada correctamente.'
            )
            # Redirige para evitar reenvío del formulario al recargar
            return redirect('encomienda_detalle', pk=enc.pk)

    # Si el form tiene errores, vuelve a mostrar con los errores
```

```

else:

    form = EncomiendaForm()    # GET: form vacío

    return render(request, 'envios/form.html', {
        'form': form,
        'titulo': 'Nueva Encomienda',
    })

```

PATRÓN POST/REDIRECT/GET (PRG)

Después de un POST exitoso SIEMPRE redirige con `redirect()`.

Si el usuario recarga la página después del redirect, no reenvía el formulario.

Si no redirigyes, el navegador pregunta '¿Reenviar los datos?' al recargar.

El Objeto Request

Toda vista recibe un objeto `HttpRequest` con toda la información de la solicitud HTTP del usuario.

Python

```

def mi_vista(request):

    # Método HTTP

    request.method          # 'GET', 'POST', 'PUT', 'DELETE'

    # Datos enviados

    request.GET              # parámetros de URL (?q=Lima&estado=TR)
    request.POST             # datos del formulario (método POST)
    request.FILES            # archivos subidos

```

```

# Usuario autenticado
request.user          # objeto User (o AnonymousUser)
request.user.username # 'juan.mendoza'
request.user.is_authenticated # True / False
request.user.email    # 'juan@encomiendas.pe'

# Sesión del usuario
request.session        # diccionario de sesión
request.session['ultima_ruta'] = 1 # guardar en sesión
ruta = request.session.get('ultima_ruta') # leer

# Meta del request
request.path           # '/encomiendas/1/'
request.get_full_path() # '/encomiendas/1/?page=2'
request.META['REMOTE_ADDR'] # IP del cliente

return HttpResponse('ok')

```

Leer parámetros GET y POST

Python

```

# — request.GET: parámetros de la URL —————
# URL: /encomiendas/?estado=TR&q=Lima&page=2

```

```

def encomienda_lista(request):
    estado = request.GET.get('estado', '') # '' si no existe
    q      = request.GET.get('q', '')
    page   = request.GET.get('page', 1)

    qs = Encomienda.objects.con_relaciones()
    if estado:
        qs = qs.filter(estado=estado)
    if q:
        from django.db.models import Q
        qs = qs.filter(
            Q(codigo__icontains=q) |
            Q(remitente__apellidos__icontains=q) |
            Q(destinatario__apellidos__icontains=q)
        )
    return render(request, 'envios/lista.html', {'encomiendas': qs})

# — request.POST: datos del formulario —————
@login_required
def encomienda_cambiar_estado(request, pk):
    enc = get_object_or_404(Encomienda, pk=pk)
    if request.method == 'POST':
        nuevo_estado = request.POST.get('estado')
        observacion = request.POST.get('observacion', '')
        try:
            empleado = Empleado.objects.get(email=request.user.email)
            enc.cambiar_estado(nuevo_estado, empleado, observacion)

```

```

        messages.success(request, f'Estado actualizado a:
{enc.get_estado_display()}')
    except ValueError as e:
        messages.error(request, str(e))
    return redirect('encomienda_detalle', pk=pk)

```

Tipos de Respuesta HTTP

Clase / Función	Status code	Cuándo usar
<code>render()</code>	200 OK	Renderizar un template con contexto — la más común
<code>redirect()</code>	302 Found	Redirigir a otra URL después de un POST
<code>HttpResponse()</code>	Configurable	Respuesta básica con texto, HTML o cualquier contenido
<code>JsonResponse()</code>	200 OK	Devolver JSON para peticiones AJAX o APIs
<code>raise Http404</code>	404 Not Found	Recurso no encontrado — mejor usar <code>get_object_or_404</code>
<code>PermissionDenied</code>	403 Forbidden	El usuario no tiene permiso para esta acción
<code>HttpResponseBadRequest</code>	400 Bad Request	Datos inválidos en la petición

Python

```

from django.http import (
    HttpResponse, HttpResponseForbidden,
    JsonResponse, Http404,
)
from django.shortcuts import render, redirect

```



```

# render - la más común
def encomienda_lista(request):
    return render(request, 'envios/lista.html', {'enc': qs})

# redirect - patrón Post/Redirect/Get
def encomienda_crear(request):
    if request.method == 'POST':
        ...
    return redirect('encomienda_detalle', pk=enc.pk)
    return render(request, 'envios/form.html', {'form': form})

# JsonResponse - endpoint AJAX para el badge del navbar
def encomienda_estado_json(request, pk):
    enc = get_object_or_404(Encomienda, pk=pk)
    return JsonResponse({
        'codigo': enc.codigo,
        'estado': enc.estado,
        'display': enc.get_estado_display(),
        'retraso': enc.tiene_retraso,
        'dias': enc.dias_en_transito,
    })

# Http404 - recurso no encontrado
def encomienda_por_codigo(request, codigo):
    try:
        enc = Encomienda.objects.get(codigo=codigo.upper())
    except Encomienda.DoesNotExist:
        raise Http404(f'No existe la encomienda {codigo}')

```

```
return render(request, 'envios/detalle.html', {'encomienda': enc})
```

```
# HttpResponse con status code personalizado
```

```
def ping(request):
```

```
    return HttpResponse('pong', status=200, content_type='text/plain')
```

Decoradores de Vistas

Los decoradores envuelven una vista añadiendo comportamiento antes o después de ejecutarla, sin modificar el código de la vista en sí. Se aplican con el símbolo @ antes de la función.

@login_required — proteger vistas

Python

```
from django.contrib.auth.decorators import login_required
```

```
# Si el usuario NO está autenticado:
```

```
# → redirige a LOGIN_URL definido en settings.py
```

```
@login_required
```

```
def dashboard(request):
```

```
    ...
```

```
# Personalizar la URL de login
```

```
@login_required(login_url='/mi-login/')
```

```
def vista_privada(request):
    ...

# En settings.py (aplica a todos los @login_required)
LOGIN_URL = '/accounts/login/'
LOGIN_REDIRECT_URL = '/'
LOGOUT_REDIRECT_URL = '/accounts/login/'
```

@require_http_methods — limitar método HTTP

```
Python

from django.views.decorators.http import (
    require_http_methods,
    require_GET,
    require_POST,
)

# Solo GET - vistas de solo lectura
@require_GET
@login_required
def encomienda_lista(request):
    ...

# Solo POST - operaciones que modifican datos
# Si se accede con GET → devuelve 405 Method Not Allowed
@require_POST
```

```

@login_required
def encomienda_cambiar_estado(request, pk):
    ...

# GET y POST
@require_http_methods(['GET', 'POST'])
@login_required
def encomienda_crear(request):
    ...

# NOTA: los decoradores se aplican de abajo hacia arriba
# @login_required    <- 2° se verifica: está autenticado?
# @require_POST      <- 1° se verifica: es POST?
def cambiar_estado(request, pk):
    ...

```

@permission_required — control de permisos

```

Python

from django.contrib.auth.decorators import (
    permission_required,
    user_passes_test,
)

# Requiere un permiso específico del sistema de permisos de Django
@permission_required('envios.add_encomienda', raise_exception=True)

```

```

def encomienda_crear(request):
    ...

# Condición personalizada para el sistema de encomiendas
def es_empleado_activo(user):
    """True si el user tiene un Empleado activo asociado"""
    return (
        user.is_authenticated and
        Empleado.objects.filter(email=user.email, estado=1).exists()
    )

@user_passes_test(es_empleado_activo, login_url='/sin-permiso/')
def registrar_envio(request):
    ...

# Verificar permisos manualmente dentro de la vista
from django.core.exceptions import PermissionDenied

@login_required
def eliminar_encomienda(request, pk):
    enc = get_object_or_404(Encomienda, pk=pk)

    # Solo se puede eliminar si está pendiente (lógica de negocio)
    if enc.estado != 'PE':
        raise PermissionDenied # → devuelve 403 Forbidden

    if request.method == 'POST':
        enc.delete()

```

```
messages.success(request, 'Encomienda eliminada.')

return redirect('encomienda_lista')

return render(request, 'envios/confirmar_eliminar.html', {'enc': enc})
```

URLs y Parámetros de Vista

Python

```
# envios/urls.py

from django.urls import path
from . import views

urlpatterns = [

    # Sin parámetros
    path('', views.dashboard, name='dashboard'),
    path('encomiendas/', views.encomienda_lista,
name='encomienda_lista'),
    path('encomiendas/nueva/', views.encomienda_crear,
name='encomienda_crear'),

    # Con parámetro entero: <int:pk>
    path('encomiendas/<int:pk>', views.encomienda_detalle,
name='encomienda_detalle'),
    path('encomiendas/<int:pk>/editar/', views.encomienda_editar,
name='encomienda_editar'),
```

```

    path('encomiendas/<int:pk>/estado/', views.encomienda_cambiar_estado,
name='encomienda_cambiar_estado'),

    # Con parámetro texto: <str:codigo>

    path('encomiendas/buscar/<str:codigo>/', views.buscar_por_codigo,
name='buscar_por_codigo'),

    # Con parámetro UUID: <uuid:uuid>

    path('api/encomiendas/<uuid:uuid>/', views.encomienda_api,
name='encomienda_api'),
]

# Convertidores disponibles:

# <int:nombre> → entero positivo
# <str:nombre> → cualquier texto (sin /)
# <slug:nombre> → letras, números, guiones, guiones bajos
# <uuid:nombre> → UUID formato xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
# <path:nombre> → texto incluyendo /

```

Generar URLs en vistas y templates

Python

```

# En vistas: reverse() y reverse_lazy()

from django.urls import reverse, reverse_lazy

def mi_vista(request):

```

```

# reverse() devuelve la URL como string
url = reverse('encomienda_detalle', kwargs={'pk': 1})
# → '/encomiendas/1/'
return redirect(url)

# reverse_lazy() para usar en atributos de clase (CBV)
class MiView(CreateView):
    success_url = reverse_lazy('encomienda_lista')

# En templates: etiqueta {% url %}
# <a href="{% url 'encomienda_lista' %}">Listado</a>
# <a href="{% url 'encomienda_detalle' enc.pk %}">Ver</a>
# <a href="{% url 'buscar_por_codigo' 'ENC-2026-001' %}">Buscar</a>
# <form method="post" action="{% url 'encomienda_cambiar_estado' enc.pk %}">

```

Paginación

Django incluye la clase `Paginator` que divide un `QuerySet` en páginas. La vista recibe el número de página de la URL (`?page=2`) y devuelve solo los registros correspondientes.

Lab 1

Vista de listado paginado con filtros

Implementar búsqueda, filtro por estado y paginación que conserva los parámetros

Python

```
# envios/views.py

from django.core.paginator import Paginator

@login_required
def encomienda_lista(request):
    qs = Encomienda.objects.con_relaciones()

    # — Filtros opcionales —————
    estado = request.GET.get('estado', '')
    q      = request.GET.get('q', '')

    if estado:
        qs = qs.filter(estado=estado)
    if q:
        from django.db.models import Q
        qs = qs.filter(
            Q(codigo__icontains=q) |
            Q(remitente__apellidos__icontains=q) |
            Q(destinatario__apellidos__icontains=q)
        )

    # — Paginación —————
    paginator = Paginator(qs, 15) # 15 por página
    page_number = request.GET.get('page', 1) # página actual
    encomiendas = paginator.get_page(page_number) # objeto Page

    # El objeto Page tiene estos atributos útiles:
    # encomiendas.number → número de página actual
```

```

# encomiendas.paginator.count      → total de registros
# encomiendas.paginator.num_pages → total de páginas
# encomiendas.has_previous()       → True/False
# encomiendas.has_next()           → True/False
# encomiendas.previous_page_number() → número página anterior
# encomiendas.next_page_number()   → número página siguiente

return render(request, 'envios/lista.html', {
    'encomiendas': encomiendas, # objeto Page (iterable)
    'estados': EstadoEnvio.choices,
    'estado_activo': estado,
    'q': q,
})

```

Template de paginación que conserva los filtros

Python

templates/partial/paginacion.html

```
{% if encomiendas.has_other_pages %}
```

```
<nav>
```

```
<ul class="pagination justify-content-center mb-0">
```

```
{% if encomiendas.has_previous %}
```

```
<li class="page-item">
```

```
{# Conservar los filtros q y estado en la URL #}
```

```
<a class="page-link"
```

```

        href="?page={{ encomiendas.previous_page_number }}&q={{ q }}&estado={{
estado_activo }}">
        &laquo; Anterior
    </a>
</li>
{% endif %}

{% for num in encomiendas.paginator.page_range %}
<li class="page-item {% if encomiendas.number == num %}active{% endif %}">
<a class="page-link"
href="?page={{ num }}&q={{ q }}&estado={{ estado_activo }}">
    {{ num }}
</a>
</li>
{% endfor %}

{% if encomiendas.has_next %}
<li class="page-item">
<a class="page-link"
href="?page={{ encomiendas.next_page_number }}&q={{ q }}&estado={{
estado_activo }}">
    Siguiete &raquo;
</a>
</li>
{% endif %}

</ul>
</nav>
{% endif %}

```

Vistas Basadas en Clases (CBV)

Las CBV organizan la lógica en métodos (get(), post()) y permiten reutilización por herencia. Las vistas genéricas de Django eliminan código repetitivo en operaciones CRUD estándar.

Vistas genéricas CRUD para Encomienda

Python

```
# envios/views_cbv.py

from django.views.generic import (
    ListView, DetailView, CreateView, UpdateView, DeleteView
)

from django.contrib.auth.mixins import LoginRequiredMixin
from django.urls import reverse_lazy
from django.contrib.messages.views import SuccessMessageMixin
from .models import Encomienda
from .forms import EncomiendaForm

# — ListView: lista paginada —————

class EncomiendaListView(LoginRequiredMixin, ListView):
    model = Encomienda
    template_name = 'envios/lista.html'
    context_object_name = 'encomiendas' # nombre en el template
    paginate_by = 15
    ordering = ['-fecha_registro']
```

```

def get_queryset(self):
    qs = Encomienda.objects.con_relaciones()
    estado = self.request.GET.get('estado')
    if estado:
        qs = qs.filter(estado=estado)
    return qs

```

```

def get_context_data(self, **kwargs):
    ctx = super().get_context_data(**kwargs)
    ctx['estados'] = EstadoEnvio.choices
    return ctx

```

— **DetailView: detalle de un registro** —————

```

class EncomiendaDetailView(LoginRequiredMixin, DetailView):
    model = Encomienda
    template_name = 'envios/detalle.html'
    context_object_name = 'encomienda'

    def get_queryset(self):
        return Encomienda.objects.con_relaciones()

    def get_context_data(self, **kwargs):
        ctx = super().get_context_data(**kwargs)
        ctx['historial'] = self.object.historial.select_related('empleado')
        return ctx

```

```

# — CreateView: formulario de creación —————

class EncomiendaCreateView(LoginRequiredMixin, SuccessMessageMixin, CreateView):
    model = Encomienda
    form_class = EncomiendaForm
    template_name = 'envios/form.html'
    success_message = 'Encomienda %(codigo)s creada correctamente.'

    def get_success_url(self):
        return reverse_lazy('encomienda_detalle', kwargs={'pk': self.object.pk})

    def form_valid(self, form):
        # Asignar el empleado antes de guardar
        form.instance.empleado_registro = self.request.user.empleado
        return super().form_valid(form)

# — UpdateView: formulario de edición —————

class EncomiendaUpdateView(LoginRequiredMixin, SuccessMessageMixin, UpdateView):
    model = Encomienda
    form_class = EncomiendaForm
    template_name = 'envios/form.html'
    success_message = 'Encomienda actualizada correctamente.'

    def get_success_url(self):
        return reverse_lazy('encomienda_detalle', kwargs={'pk': self.object.pk})

```

Registrar CBV en [urls.py](#)

Python

```
# envios/urls.py - CBV con .as_view()

from django.urls import path

from . import views_cbv

urlpatterns = [

    # .as_view() convierte la clase en una función vista
    path('', views_cbv.EncomiendaListView.as_view(),
        name='encomienda_lista'),
    path('<int:pk>/', views_cbv.EncomiendaDetailView.as_view(),
        name='encomienda_detalle'),
    path('nueva/', views_cbv.EncomiendaCreateView.as_view(),
        name='encomienda_crear'),
    path('<int:pk>/editar/', views_cbv.EncomiendaUpdateView.as_view(),
        name='encomienda_editar'),

]
```

Contexto Avanzado y Context Processors

Los Context Processors en Django son funciones que inyectan datos automáticamente en todas las plantillas, eliminando la necesidad de pasar las mismas variables en cada vista. Reciben un request y devuelven un diccionario de contexto, ideal para elementos globales como carritos de compras, menús de navegación o configuración del sitio.

Aspectos clave de Context Processors Avanzados:

- Definición: Son funciones de Python simples que reciben request y devuelven un diccionario.

- Implementación: Se definen típicamente en un archivo `context_processors.py` dentro de una aplicación y se añaden a la lista `TEMPLATES['OPTIONS']['context_processors']` en `settings.py`.
- Casos de uso avanzados: Útil para mostrar información del usuario autenticado, números de notificaciones, mensajes de flash (messages), o variables de configuración globales (`settings.DEBUG`).
- Contexto por defecto: Django incluye varios processors por defecto como `django.contrib.auth.context_processors.auth` (usuario activo) y `django.template.context_processors.request` (objeto request actual)

Lab	Context processor personalizado para encomiendas Injectar el conteo de encomiendas con retraso en el navbar de todas las páginas
-----	---

Python

```
# envios/context_processors.py
from .models import Encomienda

def estadisticas_globales(request):
    """
    Inyecta en TODOS los templates estas variables.
    Visible en el navbar sin tener que pasarlas vista por vista.
    Solo corre si el usuario está autenticado.
    """
    if not request.user.is_authenticated:
        return {}
```



```
return {  
    'nav_activas':      Encomienda.objects.activas().count(),  
    'nav_retraso':     Encomienda.objects.con_retraso().count(),  
    'nav_pendientes':  Encomienda.objects.pendientes().count(),  
}
```

Python

config/settings.py – registrar el context processor

```
TEMPLATES = [{  
    ...  
    'OPTIONS': {  
        'context_processors': [  
            'django.template.context_processors.debug',  
            'django.template.context_processors.request',  
            'django.contrib.auth.context_processors.auth',  
            'django.contrib.messages.context_processors.messages',  
            # Nuestro context processor personalizado:  
            'envios.context_processors.estadisticas_globales',  
        ],  
    },  
}]
```

Python

```
{# templates/navbar.html - usar las variables globales #}
{# Estas variables llegan a TODOS los templates automáticamente #}

<ul class="navbar-nav me-auto">
  <li class="nav-item">
    <a class="nav-link" href="{% url 'encomienda_lista' %}">
      Encomiendas
      {% if nav_retraso > 0 %}
        {# Badge rojo con el número de encomiendas retrasadas #}
        <span class="badge bg-danger rounded-pill">{{ nav_retraso }}</span>
      {% endif %}
    </a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="{% url 'dashboard' %}">
      Dashboard
      {# Mostrar total de activas #}
      <span class="badge bg-secondary">{{ nav_activas }}</span>
    </a>
  </li>
</ul>
```

Messages framework — notificaciones flash

Con bastante frecuencia en las aplicaciones web es necesario mostrar un mensaje de notificación por única vez (también conocido como «mensaje flash») al usuario después de procesar un formulario o algunos otros tipos de entrada del usuario a fin de notificarle que ha ocurrido algo en el backend.

Esta acción de emitir notificaciones es sumamente fácil de hacer gracias a las funciones incluidas en el framework web Django.

Los niveles de mensajes que Django utiliza por default son los siguientes:

Nivel	Propósito
DEBUG	Mensajes de nivel desarrollo que serán omitidos una vez que el proyecto se encuentra ejecutándose a nivel productivo.
INFO	Mensajes de información para los usuarios.
SUCCESS	Mensaje que notifica que algo ha concluido satisfactoriamente.
WARNING	Mensaje que alerta al usuario que existe una advertencia en alguna acción. No significa que haya existido un error.
ERROR	Mensaje que notifica al usuario que algo salió mal en la acción que acaba de ejecutar.

Vamos a implementarlo en nuestro proyecto:

```
Python

from django.contrib import messages

# En la vista - añadir mensajes antes del redirect
@login_required
def encomienda_crear(request):
    if request.method == 'POST':
```

```

form = EncomiendaForm(request.POST)

if form.is_valid():
    enc = form.save()
    messages.success(request, f'Encomienda {enc.codigo} creada.')
    return redirect('encomienda_detalle', pk=enc.pk)
else:
    messages.error(request, 'Corrige los errores del formulario.')

# Niveles disponibles:
messages.debug(request, 'Mensaje de depuración')
messages.info(request, 'Información general')
messages.success(request, 'Operación exitosa')
messages.warning(request, 'Atención: algo puede salir mal')
messages.error(request, 'Error: la operación falló')

{# En base.html - ya está configurado #}
{% for message in messages %}
    <div class="alert alert-{{ message.tags }} alert-dismissible fade show">
        {{ message }}
        <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    </div>
{% endfor %}

```

ENTREGABLE — VISTAS EN DJANGO

Al finalizar debes poder demostrar:

FBV (Vistas basadas en funciones)

1. dashboard() con contexto de estadísticas usando los managers personalizados.
2. encomienda_lista() con filtros GET (estado, búsqueda) y paginación de 15 registros.
3. encomienda_detalle() usando get_object_or_404() con QuerySet optimizado.
4. encomienda_crear() con patrón GET/POST, form.save(commit=False) y redirect().
5. encomienda_cambiar_estado() usando request.POST y el método cambiar_estado().

Decoradores

6. Todas las vistas protegidas con @login_required.
7. encomienda_cambiar_estado() con @require_POST.
8. Al menos una vista con verificación de permiso con PermissionDenied.

Respuestas HTTP

9. JsonResponse para un endpoint de estado de encomienda.
10. Messages flash (success, error) en operaciones CRUD.

CBV

11. EncomiendaListView con ListView + filtro en get_queryset().
12. EncomiendaDetailView con contexto del historial en get_context_data().
13. EncomiendaCreateView con form_valid() asignando el empleado.

Context processor

14. estadisticas_globales() registrado en settings.py.
15. Navbar muestra el badge de encomiendas con retraso.

Repositorio actualizado en GitHub con views.py, views_cbv.py y context_processors.py.

Formularios en Django

Cuando trabajamos con una aplicación web interactiva, no sólo queremos proporcionar datos a los usuarios, sino también aceptar datos de ellos, ya sea para personalizar las respuestas que estamos generando o para permitirles enviar datos al sitio. Al navegar por Internet, seguro que has utilizado formularios. Tanto si estás conectado a tu cuenta de banca por Internet, navegando por la web con un navegador, publicando un mensaje en las redes sociales o escribiendo un correo electrónico a un cliente de correo electrónico en línea, en todos estos casos, estás introduciendo datos en un formulario. Un formulario se compone de entradas que definen pares clave-valor de datos para enviar al servidor. Por ejemplo, al iniciar sesión en un sitio web, los datos que se envían tendrían las claves nombre de usuario y contraseña, con los valores de su nombre de usuario y su contraseña, respectivamente. Hablaremos de los diferentes tipos de entradas con más detalle en una próxima sección. Cada entrada del formulario tiene un nombre, y así es como se identifican sus datos en el lado del servidor (en una vista de Django). Puede haber múltiples entradas con el mismo nombre, cuyos datos están disponibles en una lista que contiene todos los valores publicados con este nombre - por ejemplo, una lista de casillas de verificación con permisos para aplicar a los usuarios. Cada casilla de verificación tendría el mismo nombre pero un valor diferente. El formulario tiene atributos que especifican a qué URL debe enviar los datos el navegador y qué método debe utilizar para enviarlos (los navegadores sólo admiten GET o POST).

Creación de formularios en Django

Puedes crear formularios en Django manualmente con la clase `Form` o generados automáticamente a partir de modelos usando `ModelForm`.

Para crear un formulario en Django, define una clase `Form` en tu archivo `forms.py`. Esta clase especifica los campos del formulario

- `forms.Form`: una clase estándar para formularios que no están vinculados directamente a un modelo de base de datos (por ejemplo, formularios de búsqueda o de contacto). Cada campo se define manualmente.
- `forms.ModelForm`: una clase auxiliar que genera automáticamente campos de formulario a partir de un modelo de Django ya existente. También incluye un método `.save()` para escribir datos directamente en la base de datos

Lab 4.1 — Formulario para registrar una encomienda

Lab 4.1

EncomiendaForm con ModelForm

Crea el formulario para registrar envíos nuevos

Crea el archivo `envios/forms.py`:

Python

```
# envios/forms.py

from django import forms
from .models import Encomienda
from clientes.models import Cliente
from rutas.models import Ruta
from config.choices import EstadoGeneral

class EncomiendaForm(forms.ModelForm):

    """Formulario para registrar una nueva encomienda"""

    class Meta:

        model = Encomienda

        fields = [

            'codigo', 'descripcion', 'peso_kg', 'volumen_cm3',

            'remitente', 'destinatario', 'ruta',

            'costo_envio', 'fecha_entrega_est', 'observaciones',

        ]
```

```

        widgets = {
            'codigo': forms.TextInput(attrs={'class': 'form-control'}),
            'descripcion': forms.Textarea(attrs={'class': 'form-control',
'rows': 3})),
            'peso_kg': forms.NumberInput(attrs={'class': 'form-control',
'step': '0.01'}),
            'volumen_cm3': forms.NumberInput(attrs={'class': 'form-control',
'step': '0.01'}),
            'remitente': forms.Select(attrs={'class': 'form-select'}),
            'destinatario': forms.Select(attrs={'class': 'form-select'}),
            'ruta': forms.Select(attrs={'class': 'form-select'}),
            'costo_envio': forms.NumberInput(attrs={'class': 'form-control',
'step': '0.01'}),
            'fecha_entrega_est': forms.DateInput(
                attrs={'class': 'form-control', 'type': 'date'}
            ),
            'observaciones': forms.Textarea(attrs={'class': 'form-control',
'rows': 2})),
        }

        labels = {
            'codigo': 'Código de encomienda',
            'peso_kg': 'Peso (kg)',
            'volumen_cm3': 'Volumen (cm³)',
            'fecha_entrega_est': 'Fecha estimada de entrega',
        }

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

```



```

# Mostrar solo clientes activos

self.fields['remitente'].queryset = Cliente.objects.ativos()

self.fields['destinatario'].queryset = Cliente.objects.ativos()

# Mostrar solo rutas activas

self.fields['ruta'].queryset = Ruta.objects.activas()


def clean(self):

    """Validaciones adicionales a nivel de formulario"""

    cleaned = super().clean()

    remitente = cleaned.get('remitente')

    destinatario = cleaned.get('destinatario')

    if remitente and destinatario and remitente == destinatario:

        raise forms.ValidationError(

            'El remitente y el destinatario no pueden ser la misma persona.'

        )

    return cleaned

```

Elementos clave de los formularios Django

Elemento	Descripción
{% csrf_token %}	Token de seguridad obligatorio en todo formulario POST. Protege contra ataques CSRF
{{ form.field.errors }}	Muestra los errores de un campo específico

<code>{{ form.non_field_errors }}</code>	Muestra errores que no pertenecen a ningún campo
<code>form.is_valid()</code>	Valida todos los campos y ejecuta <code>clean()</code>
<code>form.cleaned_data</code>	Diccionario con los datos validados y limpios
<code>form.save()</code>	Guarda el formulario en la BD (solo <code>ModelForm</code>)

Template del formulario de encomienda

Ahora vamos a crear este archivo `templates/envios/form.html`

Python

```
{% extends 'base.html' %}

{% block title %}{{ titulo }} - Encomiendas{% endblock %}

{% block content %}

<div class="d-flex justify-content-between align-items-center mb-4">
    <h2 class="h3"><i class="fas fa-plus me-2"></i>{{ titulo }}</h2>
    <a href="{% url 'encomienda_lista' %}" class="btn btn-outline-secondary">
        <i class="fas fa-arrow-left me-1"></i>Volver
    </a>
</div>

<div class="card shadow">
    <div class="card-body">
        <form method="post" id="encomiendaForm">
            {% csrf_token %}

            {% if form.non_field_errors %}
```

```

<div class="alert alert-danger">
    {% for error in form.non_field_errors %}<p>{{ error }}</p>{% endfor %}
</div>

{% endif %}

<!-- Código y descripción -->
<div class="row mb-3">
    <div class="col-md-4">
        <label class="form-label">Código *</label>
        {{ form.codigo }}
        {% if form.codigo.errors %}
            <div class="invalid-feedback d-block">{{ form.codigo.errors|join:", "
}}</div>
        {% endif %}
    </div>
    <div class="col-md-8">
        <label class="form-label">Descripción *</label>
        {{ form.descripcion }}
    </div>
</div>

<!-- Remitente y Destinatario -->
<div class="row mb-3">
    <div class="col-md-6">
        <label class="form-label">Remitente *</label>
        {{ form.remitente }}
    </div>
    <div class="col-md-6">
        <label class="form-label">Destinatario *</label>
        {{ form.destinatario }}
    </div>
</div>

```

```

    {% if form.remitente.errors %}

    <div class="invalid-feedback d-block">{{ form.remitente.errors }}</div>

    {% endif %}

</div>

<div class="col-md-6">

    <label class="form-label">Destinatario *</label>

    {{ form.destinatario }}

</div>

</div>

<!-- Ruta y Costo -->

<div class="row mb-3">

    <div class="col-md-6">

        <label class="form-label">Ruta *</label>

        {{ form.ruta }}

    </div>

    <div class="col-md-3">

        <label class="form-label">Peso (kg) *</label>

        {{ form.peso_kg }}

    </div>

    <div class="col-md-3">

        <label class="form-label">Costo (S/) *</label>

        {{ form.costo_envio }}

    </div>

</div>

```

```

<div class="d-flex justify-content-end mt-4">

    <button type="button" class="btn btn-outline-secondary me-2"
onclick="history.back()">Cancelar</button>

    <button type="submit" class="btn btn-primary">

        <i class="fas fa-save me-1"></i>Guardar Encomienda

    </button>

</div>

</form>

</div>

</div>

{% endblock %}

```

Renderizado de formularios: tres opciones

El renderizado de formularios en Django implica definir una clase Form o ModelForm en forms.py, instanciarla en la vista y pasarla al template. Se renderiza en HTML usando etiquetas como `{{ form.as_p }}` para métodos rápidos, o desglosando campos manualmente (`{{ form.campo }}`) para mayor personalización.

Métodos de Renderizado en Templates

Método	Código en el template	Etiqueta HTML generada	Cuándo usarlo
<code>form.as_p()</code>	<code>{{ form.as_p }}</code>	<code><p></code> por campo	Prototipado rápido
<code>form.as_table()</code>	<code>{{ form.as_table }}</code>	<code><tr>/<th>/<td></code> por campo	Layouts tabulares simples
<code>form.as_ul()</code>	<code>{{ form.as_ul }}</code>	<code></code> por campo	Poco frecuente en la práctica

Campo por campo	{{ form.campo }}	Control total del HTML	Producción con Bootstrap
-----------------	------------------	------------------------	--------------------------

Método 1: form.as_p()

Envuelve cada campo en una etiqueta <p>. Es la opción más simple: una sola línea de código genera el formulario completo con labels, inputs y errores.

Código en el template

Python

```
{% extends 'base.html' %}

{% block content %}

<form method="post">

    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Guardar encomienda</button>

</form>

{% endblock %}
```

HTML que Django genera automáticamente

Django expande {{ form.as_p }} en este HTML:

HTML

```
<p>

  <label for="id_codigo">Código de encomienda:</label>

  <input type="text" name="codigo" id="id_codigo" maxlength="20">

</p>

<p>

  <label for="id_remitente">Remitente:</label>

  <select name="remitente" id="id_remitente">

    <option value="1">Carlos Ramirez Torres</option>

    <option value="2">Ana Flores Diaz</option>

  </select>

</p>

<p>

  <ul class="errorlist">

    <li>El campo es obligatorio.</li>

  </ul>

  <label for="id_peso_kg">Peso (kg):</label>

  <input type="number" name="peso_kg" id="id_peso_kg">

</p>
```

VISTA PREVIA GENERADA

Automático

Código de encomienda:

ENC-2026-001

Remitente:

Carlos Ramirez Torres

Destinatario:

Ana Flores Diaz

Peso (kg):

3.5

Guardar

Django envuelve cada campo en un `<p>` con su `<label>` y errores automáticamente

Ventajas y limitaciones

Ventajas	Limitaciones
Una sola línea de código genera todo el form	Los inputs no tienen clases de Bootstrap (form-control)
Incluye labels y errores automáticamente	Sin control sobre el layout (columnas, grupos)
Ideal para prototipos rápidos o apps internas	Difícil de personalizar el diseño
No requiere conocer los nombres de los campos	No es adecuado para interfaces profesionales

Método 2: `form.as_table()`

Genera cada campo como una fila de tabla `<tr>` con el label en `<th>` y el input en `<td>`. Requiere que tú agregues las etiquetas `<table>` y `</table>` manualmente.

Python

```
{% block content %}

<form method="post">

    {% csrf_token %}

    <table>

        {{ form.as_table }}

    </table>

    <button type="submit">Guardar</button>

</form>

{% endblock %}
```

HTML que Django genera automáticamente

HTML

```
<tr>

    <th><label for="id_codigo">Código:</label></th>

    <td><input type="text" name="codigo" id="id_codigo"></td>

</tr>
```

```

<tr>
  <th><label for="id_remitente">Remitente:</label></th>
  <td>
    <select name="remitente" id="id_remitente">
      <option value="1">Carlos Ramirez Torres</option>
    </select>
  </td>
</tr>
<tr>
  <th><label for="id_peso_kg">Peso (kg):</label></th>
  <td>
    <ul class="errorlist"><li>Este campo es requerido.</li></ul>
    <input type="number" name="peso_kg" id="id_peso_kg">
  </td>
</tr>

```

NOTA

Los errores aparecen DENTRO de la celda <td>, antes del input. Django los coloca en una lista <ul class="errorlist">.

Recuerda agregar las etiquetas <table> y </table> en el template, ya que form.as_table() solo genera las filas <tr>.

Método 3: `form.as_ul()`

Genera cada campo como un elemento `<li>` de una lista. Requiere que tú agregues las etiquetas `<ul>` y `</ul>` manualmente. Es semánticamente limpio pero poco usado en la práctica porque sigue sin tener clases Bootstrap.

Código en el template

Python

```
{% block content %}

<form method="post">

    {% csrf_token %}

    <ul>

        {{ form.as_ul }}

    </ul>

    <button type="submit">Guardar</button>

</form>

{% endblock %}
```

HTML que Django genera automáticamente

HTML

```
<li>

    <label for="id_codigo">Codigo:</label>

    <input type="text" name="codigo" id="id_codigo">

</li>

<li>
```

```

<label for="id_remitente">Remitente:</label>

<select name="remitente" id="id_remitente">

    <option value="1">Carlos Ramirez Torres</option>

</select>

</li>

```

Método 4: Campo por campo (recomendado)

Este es el método profesional. En lugar de dejar que Django genere el HTML completo, accedemos a cada parte del formulario individualmente: el label, el input, los errores. Esto nos da control total para aplicar clases de Bootstrap y organizar el layout en columnas.

Variables disponibles por campo

Variable	Qué genera	Ejemplo de uso
{{ form.campo }}	Solo el input HTML con sus atributos	{{ form.codigo }}
{{ form.campo.label }}	El texto del label (sin etiqueta HTML)	{{ form.codigo.label }}
{{ form.campo.label_tag }}	La etiqueta <label for="..."> completa	{{ form.codigo.label_tag }}
{{ form.campo.errors }}	Lista de errores del campo	{{ form.codigo.errors }}
{{ form.campo.id_for_label }}	El valor del atributo id del input	{{ form.codigo.id_for_label }}
{{ form.campo.help_text }}	El texto de ayuda definido en el Form	{{ form.codigo.help_text }}
{{ form.non_field_errors }}	Errores globales (del método clean())	{{ form.non_field_errors }}

Estructura completa para el sistema de encomiendas

templates/envios/form.html — campo por campo (producción)

Python

```
{% extends 'base.html' %}

{% block title %}{{ titulo }} - Encomiendas{% endblock %}

{% block content %}

<div class="d-flex justify-content-between align-items-center mb-4">

    <h2 class="h3">{{ titulo }}</h2>

    <a href="{% url 'encomienda_lista' %}" class="btn btn-outline-secondary">Volver</a>

</div>

<div class="card shadow">

    <div class="card-body">

        <form method="post">

            {% csrf_token %}

            <!-- Errores globales (ej: remitente == destinatario) -->

            {% if form.non_field_errors %}

            <div class="alert alert-danger">

                {% for error in form.non_field_errors %}

                    <p class="mb-0">{{ error }}</p>

                {% endfor %}

            </div>

            {% endif %}

        </form>

    </div>

</div>
```

```

<!-- Fila 1: Código y Descripción ----- -->
<div class="row mb-3">
  <div class="col-md-4">
    <label class="form-label">{{ form.codigo.label }} *</label>
    {{ form.codigo }}
    {% if form.codigo.errors %}
    <div class="invalid-feedback d-block">
      {{ form.codigo.errors|join:", " }}
    </div>
    {% endif %}
  </div>
  <div class="col-md-8">
    <label class="form-label">{{ form.descripcion.label }} *</label>
    {{ form.descripcion }}
  </div>
</div>

<!-- Fila 2: Remitente y Destinatario ----- -->
<div class="row mb-3">
  <div class="col-md-6">
    <label class="form-label">{{ form.remitente.label }} *</label>
    {{ form.remitente }}
    {% if form.remitente.errors %}
    <div class="invalid-feedback d-block">

```

```

        {{ form.remitente.errors }}
    </div>

    {% endif %}
</div>

<div class="col-md-6">

    <label class="form-label">{{ form.destinatario.label }} *</label>

    {{ form.destinatario }}

</div>
</div>

<!-- Fila 3: Ruta, Peso y Costo ----- -->
<div class="row mb-3">

    <div class="col-md-6">

        <label class="form-label">{{ form.ruta.label }} *</label>

        {{ form.ruta }}

    </div>

    <div class="col-md-3">

        <label class="form-label">{{ form.peso_kg.label }} *</label>

        {{ form.peso_kg }}

    </div>

    <div class="col-md-3">

        <label class="form-label">{{ form.costo_envio.label }} *</label>

        {{ form.costo_envio }}

    </div>
</div>

```

```

<!-- Fila 4: Fecha estimada y Observaciones ----- -->
<div class="row mb-3">
  <div class="col-md-4">
    <label class="form-label">{{ form.fecha_entrega_est.label }}</label>
    {{ form.fecha_entrega_est }}
    {% if form.fecha_entrega_est.errors %}
    <div class="invalid-feedback d-block">
      {{ form.fecha_entrega_est.errors }}
    </div>
    {% endif %}
  </div>
  <div class="col-md-8">
    <label class="form-label">{{ form.observaciones.label }}</label>
    {{ form.observaciones }}
  </div>
</div>

<!-- Botones ----- -->
<div class="d-flex justify-content-end mt-4">
  <button type="button" class="btn btn-outline-secondary me-2"
    onclick="history.back()">Cancelar</button>
  <button type="submit" class="btn btn-primary">
    Guardar Encomienda
  </button>
</div>

```



```
</div>

</form>

</div>

</div>

{% endblock %}
```

Por qué los inputs tienen clases Bootstrap

Si el formulario tiene `widgets` configurados en `forms.py`, los inputs ya incluyen las clases `form-control` o `form-select` cuando se renderizan:

Python

```
# envios/forms.py

class EncomiendaForm(forms.ModelForm):

    class Meta:

        model = Encomienda

        fields = ['codigo', 'descripcion', 'peso_kg', 'remitente', ...]

        widgets = {

            # Clase form-control -> todos los inputs de texto/numero

            'codigo': forms.TextInput(attrs={'class': 'form-control'}),

            'descripcion': forms.Textarea(attrs={'class': 'form-control', 'rows':
3}),

            'peso_kg': forms.NumberInput(attrs={'class': 'form-control',
'step': '0.01'}),
```

```

# Clase form-select -> todos los despleguables (ForeignKey)

'remitente': forms.Select(attrs={'class': 'form-select'}),

'ruta':      forms.Select(attrs={'class': 'form-select'}),

}

```

Cuando el template dice {{ form.codigo }}

Django genera exactamente esto:

```
# <input type="text" name="codigo" class="form-control"
```

```
#      maxlength="20" id="id_codigo">
```

Resumen — ¿Cuándo usar cada método?

Situación	Método recomendado
Prototipo rápido o formulario interno sin diseño	form.as_p()
Demostración rápida en clase o sh shell de Django	form.as_p() o form.as_table()
Sistema con Bootstrap en producción	Campo por campo
Formulario con layout en columnas (row/col-md-6)	Campo por campo
Errores visibles con estilos Bootstrap (invalid-feedback)	Campo por campo
Formulario del proyecto de encomiendas	Campo por campo



Para el Sistema de Encomiendas siempre usaremos campo por campo.

Los métodos `as_p` / `as_table` / `as_ul` existen y es importante conocerlos, pero en un sistema real con Bootstrap el control campo por campo es lo estándar.

Recuerda: el widget del `forms.py` define el HTML del input, y el template define el layout de cómo se organizan visualmente.

Django Admin Site

El administrador de Django proporciona una interfaz basada en web para crear y administrar objetos de modelo de base de datos.

Configuración avanzada del Admin para encomiendas

Actualiza `envios/admin.py` con configuraciones profesionales:

Python

```
# envios/admin.py
from django.contrib import admin
from django.utils.html import format_html
from .models import Empleado, Encomienda, HistorialEstado
from config.choices import EstadoEnvio

@admin.register(Encomienda)
class EncomiendaAdmin(admin.ModelAdmin):
    # Columnas visibles en el listado
    list_display = ('codigo', 'remitente_nombre', 'destinatario_nombre',
                   'ruta', 'estado_badge', 'peso_kg', 'fecha_registro')

    # Filtros laterales
    list_filter = ('estado', 'ruta', 'fecha_registro')

    # Búsqueda
    search_fields = ('codigo', 'remitente__apellidos',
                   'destinatario__apellidos', 'remitente__nro_doc')

    # Campos de solo lectura
    readonly_fields = ('codigo', 'fecha_registro', 'fecha_entrega_real')

    # Ordenamiento por defecto
    ordering = ('-fecha_registro',)

    # Registros por página
    list_per_page = 20

    # Organizar los campos en secciones (fieldsets)
    fieldsets = (
        ('Identificación', {
            'fields': ('codigo', 'descripcion', 'peso_kg', 'volumen_cm3')
        }),
        ('Partes', {
```

```

        'fields': ('remitente', 'destinatario', 'ruta', 'empleado_registro')
    )),
    ('Estado y fechas', {
        'fields': ('estado', 'costo_envio',
                    'fecha_registro', 'fecha_entrega_est', 'fecha_entrega_real')
    )),
    ('Notas', {
        'classes': ('collapse',), # sección colapsable
        'fields': ('observaciones',)
    )),
)

# Método personalizado para mostrar nombre del remitente
def remitente_nombre(self, obj):
    return obj.remitente.nombre_completo
    remitente_nombre.short_description = 'Remitente'

def destinatario_nombre(self, obj):
    return obj.destinatario.nombre_completo
    destinatario_nombre.short_description = 'Destinatario'

# Método con HTML: muestra el estado con color
def estado_badge(self, obj):
    colores = {
        'PE': '#6c757d', # gris - pendiente
        'TR': '#0d6efd', # azul - en tránsito
        'DE': '#fd7e14', # naranja - en destino
        'EN': '#198754', # verde - entregado
        'DV': '#dc3545', # rojo - devuelto
    }
    color = colores.get(obj.estado, '#6c757d')
    return format_html(
        '<span style="background:{};color:white;padding:2px 8px;border-radius:4px">{}</span>',
        color, obj.get_estado_display()
    )
    estado_badge.short_description = 'Estado'

@admin.register(Empleado)
class EmpleadoAdmin(admin.ModelAdmin):
    list_display = ('codigo', 'apellidos', 'nombres', 'cargo', 'email',
                    'estado')
    list_filter = ('cargo', 'estado')

```

```
search_fields = ('codigo', 'apellidos', 'nombres', 'email')
```

```
@admin.register(HistorialEstado)
```

```
class HistorialEstadoAdmin(admin.ModelAdmin):  
    list_display = ('encomienda', 'estado_anterior', 'estado_nuevo',  
                   'empleado', 'fecha_cambio')  
    readonly_fields = ('encomienda', 'estado_anterior', 'estado_nuevo',  
                      'empleado', 'fecha_cambio')  
    list_filter = ('estado_nuevo',)  
    ordering = ('-fecha_cambio',)
```

Personalizar el título del Admin

Python

```
# config/settings.py o bien en cualquier apps.py cargado  
# También puedes hacerlo en envios/apps.py o en config/urls.py
```

```
# config/urls.py
```

```
from django.contrib import admin
```

```
admin.site.site_header = 'Sistema de Gestión de Encomiendas'
```

```
admin.site.site_title = 'Encomiendas Admin'
```

```
admin.site.index_title = 'Panel de Administración'
```

CREAR SUPERUSUARIO EN DOCKER

Para acceder al Admin de Django necesitas un superusuario:

```
docker compose exec web python manage.py createsuperuser
```

Luego accede en: <http://localhost:8000/admin>

Ingresa el usuario y contraseña que acabas de crear.

Django Authentication

Django incluye un sistema de autenticación completo: modelo de usuario, login/logout, protección de vistas y gestión de permisos. Para el sistema de encomiendas, solo los empleados autenticados pueden acceder.

Configuración de autenticación

Agrega estas variables en `config/settings.py` para controlar el flujo de login:

Python

```
# config/settings.py

# URL a donde redirigir si el usuario no está autenticado
LOGIN_URL = '/accounts/login/'

# URL a donde redirigir después de un login exitoso
LOGIN_REDIRECT_URL = '/'

# URL a donde redirigir después de logout
LOGOUT_REDIRECT_URL = '/accounts/login/'
```


Vistas de autenticación

Crea el archivo `envios/views_auth.py` con las vistas de login y logout:

Python

```
# envios/views_auth.py (o accounts/views.py si tienes esa app)

from django.shortcuts import render, redirect

from django.contrib.auth import authenticate, login, logout

from django.contrib.auth.decorators import login_required

from django.contrib import messages

from django.contrib.auth.forms import AuthenticationForm


def login_view(request):

    """Vista para el login de empleados"""

    # Si ya está autenticado, redirigir al dashboard

    if request.user.is_authenticated:

        return redirect('dashboard')

    if request.method == 'POST':

        form = AuthenticationForm(request, data=request.POST)

        if form.is_valid():

            username = form.cleaned_data.get('username')

            password = form.cleaned_data.get('password')

            user = authenticate(username=username, password=password)

            if user is not None:

                login(request, user)
```

```

        messages.success(request, f'Bienvenido, {user.get_full_name() or
user.username}!')

        # Redirigir a la página solicitada o al dashboard
        next_page = request.GET.get('next', 'dashboard')
        return redirect(next_page)

    else:

        messages.error(request, 'Usuario o contraseña incorrectos.')

    else:

        messages.error(request, 'Por favor corrige los errores.')

    else:

        form = AuthenticationForm()

    return render(request, 'accounts/login.html', {'form': form})

def logout_view(request):

    """Cierra la sesión del usuario"""

    logout(request)

    messages.info(request, 'Has cerrado sesión correctamente.')

    return redirect('login')

@login_required
def perfil_view(request):

    """Perfil del empleado autenticado"""

    return render(request, 'accounts/perfil.html', {'user': request.user})

```

Template de login

Crea `templates/accounts/login.html`:

Python

```
{% extends 'base.html' %}

{% load static %}

{% block title %}Iniciar Sesión - Encomiendas{% endblock %}

{% block content %}

<div class="row justify-content-center mt-5">
    <div class="col-md-5">
        <div class="text-center mb-4">
            <i class="fas fa-box-open fa-4x text-primary mb-3"></i>
            <h2>Sistema de Encomiendas</h2>
            <p class="text-muted">Ingresa tus credenciales para continuar</p>
        </div>
        <div class="card shadow">
            <div class="card-body p-4">
                {% if form.errors %}
                <div class="alert alert-danger">
                    <i class="fas fa-exclamation-triangle me-2"></i>
```

```

    Usuario o contraseña incorrectos

</div>

{% endif %}

<form method="post" action="{% url 'login' %}">
    {% csrf_token %}

    <div class="mb-3">
        <label class="form-label">Usuario</label>
        <div class="input-group">
            <span class="input-group-text"><i class="fas fa-user"></i></span>
            <input type="text" name="username" class="form-control"
                placeholder="Tu usuario" required autofocus>
        </div>
    </div>

    <div class="mb-4">
        <label class="form-label">Contraseña</label>
        <div class="input-group">
            <span class="input-group-text"><i class="fas fa-lock"></i></span>
            <input type="password" name="password" class="form-control"
                placeholder="Tu contraseña" required>
        </div>
    </div>

    <div class="d-grid">
        <button type="submit" class="btn btn-primary btn-lg">
            <i class="fas fa-sign-in-alt me-2"></i>Iniciar Sesión

```

```
        </button>

    </div>

    <input type="hidden" name="next" value="{{ next }}">

</form>

</div>

</div>

</div>

</div>

{% endblock %}
```

Proteger vistas con `@login_required`

El decorador `@login_required` redirige automáticamente a la página de login si el usuario no está autenticado:

Python

```
from django.contrib.auth.decorators import login_required

# Forma 1: decorator en la función

@login_required
def dashboard(request):
    ...

# Forma 2: en urls.py (sin modificar la vista)
```

```

from django.contrib.auth.decorators import login_required

urlpatterns = [
    path('', login_required(views.dashboard), name='dashboard'),
]

# Forma 3: verificar manualmente dentro de la vista
def mi_vista(request):
    if not request.user.is_authenticated:
        from django.shortcuts import redirect
        return redirect('login')
    ...

# En los templates: verificar si el usuario está autenticado
{% if user.is_authenticated %}
    <a href="{% url 'perfil' %}">{{ user.username }}</a>
    <a href="{% url 'logout' %}">Cerrar sesión</a>
{% else %}
    <a href="{% url 'login' %}">Iniciar sesión</a>
{% endif %}

```

Registrar URLs de autenticación

Django incluye vistas de login/logout listas para usar. Solo necesitas configurar los templates:

```
Python

# config/urls.py

from django.urls import path, include

urlpatterns = [

    # Opción A: URLs built-in de Django (login, logout, cambiar contraseña)

    path('accounts/', include('django.contrib.auth.urls')),

    # Opción B: Vistas propias (más control)

    path('login/', views_auth.login_view, name='login'),
    path('logout/', views_auth.logout_view, name='logout'),
    path('perfil/', views_auth.perfil_view, name='perfil'),

]

# Si usas las URLs de Django, debes crear el template en:

# templates/registration/login.html

# (Django busca esa ruta por defecto)
```

Sesiones en Django

Las sesiones permiten almacenar datos del usuario entre peticiones HTTP. Django guarda las sesiones en la base de datos por defecto y usa una cookie en el navegador para identificarlas.

Cómo funcionan las sesiones

Aspecto	Descripción
Almacenamiento	BD (por defecto), caché, archivo o cookies firmadas
Identificación	Cookie sessionid en el navegador del usuario
Acceso	request.session (activo en toda la aplicación)
Duración	Configurable: expira al cerrar el navegador o en X segundos
Seguridad	El contenido se guarda en el servidor, no en la cookie

Uso práctico en el sistema de encomiendas

Uso de request.session

Python

```
# En cualquier vista: guardar datos en la sesión

def seleccionar_ruta(request):

    if request.method == 'POST':

        ruta_id = request.POST.get('ruta_id')

        # Guardar la ruta seleccionada en sesión

        request.session['ruta_seleccionada'] = ruta_id

        request.session['ultimo_filtro_estado'] = request.POST.get('estado', 'PE')

        ...

# En otra vista: leer datos de la sesión

def nueva_encomienda(request):

    # Recuperar la ruta guardada previamente
```



```
ruta_id = request.session.get('ruta_seleccionada')

if ruta_id:

    from rutas.models import Ruta

    ruta_preseleccionada = Ruta.objects.get(pk=ruta_id)

    ...

# Eliminar un valor de sesión

del request.session['ruta_seleccionada']

# Eliminar toda la sesión

request.session.flush()

# Verificar si existe una clave

if 'ultimo_filtro_estado' in request.session:

    estado = request.session['ultimo_filtro_estado']
```

Configuración de sesiones en [settings.py](#)

Python

```
# config/settings.py

# Motor de sesiones (base de datos por defecto)
```

```
SESSION_ENGINE = 'django.contrib.sessions.backends.db'

# Expirar cuando el navegador se cierra (False = no expira)
SESSION_EXPIRE_AT_BROWSER_CLOSE = False

# Tiempo de vida de la sesión en segundos (8 horas = jornada laboral)
SESSION_COOKIE_AGE = 60 * 60 * 8

# Solo enviar la cookie por HTTPS (activar en producción)
SESSION_COOKIE_SECURE = False # True en producción

# Nombre de la cookie de sesión
SESSION_COOKIE_NAME = 'encomiendas_session'

# Asegurarse de tener sessions en INSTALLED_APPS:
INSTALLED_APPS = [
    ...
    'django.contrib.sessions', # <- requerido
    ...
]

# Y en MIDDLEWARE:
MIDDLEWARE = [
    ...
    'django.contrib.sessions.middleware.SessionMiddleware', # <- requerido
```

```
...  
]
```

Flujo completo: Login → Dashboard → Logout

Lab

Probar el flujo de autenticación

Verificar login, protección de vistas y logout en Docker

Python

1. Levantar los contenedores

```
docker compose up -d
```

2. Aplicar migraciones (incluye tabla de sesiones)

```
docker compose exec web python manage.py migrate
```

3. Crear un superusuario para pruebas

```
docker compose exec web python manage.py createsuperuser
```

4. Verificar la tabla de sesiones en psql

```
docker compose exec db psql -U encomiendas_user -d encomiendas_db -c  
'\dt django_session;'
```

5. Abrir el navegador y probar el flujo:

```
#      http://localhost:8000/          -> redirige a login (no autenticado)
#      http://localhost:8000/login/    -> formulario de login
#      Ingresar credenciales           -> redirige al dashboard
#      http://localhost:8000/admin/    -> panel de administración
#      http://localhost:8000/logout/   -> cierra sesión y redirige a login
```

Navbar con estado de autenticación

Actualiza `templates/navbar.html` para mostrar el menú correcto según si el usuario está autenticado:

Python

```
<nav class="navbar navbar-expand-lg navbar-dark bg-primary">
  <div class="container">
    <a class="navbar-brand" href="{% url 'dashboard' %}">
      <i class="fas fa-shipping-fast me-2"></i>Encomiendas
    </a>

    {% if user.is_authenticated %}
    <div class="collapse navbar-collapse">
      <ul class="navbar-nav me-auto">
        <li class="nav-item">
```

```

<a class="nav-link" href="{% url 'dashboard' %}">Dashboard</a>
</li>

<li class="nav-item">
<a class="nav-link" href="{% url 'encomienda_lista' %}">Encomiendas</a>
</li>

<li class="nav-item">
<a class="nav-link" href="{% url 'encomienda_crear' %}">Nueva</a>
</li>
</ul>

<div class="d-flex">
<span class="navbar-text me-3">
<i class="fas fa-user me-1"></i>{{ user.username }}
</span>
<a href="{% url 'logout' %}" class="btn btn-outline-light btn-sm">
<i class="fas fa-sign-out-alt me-1"></i>Salir
</a>
</div>
</div>

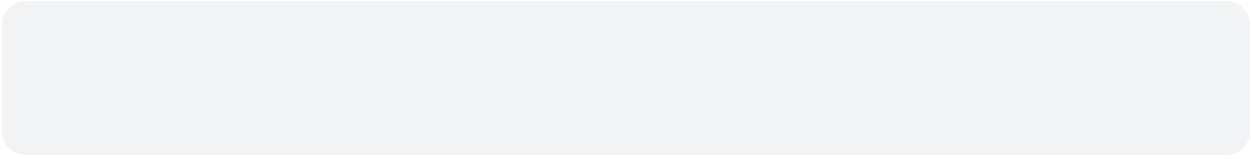
{% else %}
<a href="{% url 'login' %}" class="btn btn-outline-light">
<i class="fas fa-sign-in-alt me-1"></i>Iniciar sesión
</a>

{% endif %}

</div>
</nav>

```

—



Estructura final del proyecto — Sesión 4

encomiendas/	
└─ config/	
└─ settings.py	<- TEMPLATES, STATIC, LOGIN_URL, SESSION
└─ urls.py	<- rutas globales + include de apps
└─ envios/	
└─ views.py	<- dashboard, lista, detalle, crear, cambiar estado
└─ views_auth.py	<- login view, logout view, perfil view
└─ forms.py	<- EncomiendaForm
└─ urls.py	<- rutas de la app envios
└─ admin.py	<- EncomiendaAdmin con badges y fieldsets
└─ templates/	
└─ base.html	<- plantilla base con Bootstrap
└─ navbar.html	<- navbar principal
└─ index.html	
└─ dashboard.html	<- tarjetas + tabla de últimas
└─ lista.html	<- tabla paginada con filtros
└─ detalle.html	<- info completa + historial de estados
└─ form.html	<- formulario crear/editar
└─ accounts/	
└─ login.html	<- página de login
└─ register.html	<- perfil del empleado
└─ static/	
└─ css/styles.css	
└─ js/main.js	
└─ Dockerfile	
└─ docker-compose.yml	
└─ .env	

Al finalizar las semanas 6-7 debes poder demostrar:

URLs, Views y Templates

1. Dashboard funcional con contadores de encomiendas activas, en tránsito y con retraso.
2. Lista de encomiendas con paginación (15 por página) y filtro por estado.
3. Detalle de encomienda mostrando info completa y el historial de estados.
4. Navbar con links al dashboard, listado y creación.

Formularios

5. Formulario de nueva encomienda con validación client-side y server-side.
6. EncomiendaForm filtra solo clientes y rutas activos.
7. Mensajes flash de éxito/error en operaciones CRUD.

Admin Site

8. EncomiendaAdmin con badges de color por estado y fieldsets.
9. Título del admin personalizado con el nombre del sistema.

Autenticación y Sesiones

10. Login y logout funcionando correctamente.
11. Todas las vistas protegidas con `@login_required`.
12. Navbar muestra el usuario logueado y el botón de salir.
13. Redirige a login cuando se intenta acceder sin autenticar.

Repositorio actualizado en GitHub con todos los archivos nuevos.